



US012399630B2

(12) **United States Patent**
Binfet et al.

(10) **Patent No.:** **US 12,399,630 B2**

(45) **Date of Patent:** **Aug. 26, 2025**

(54) **PEAK POWER MANAGEMENT PRIORITY OVERRIDE**

(71) Applicant: **Micron Technology, Inc.**, Boise, ID (US)

(72) Inventors: **Jeremy Binfet**, Boise, ID (US); **Liang Yu**, Boise, ID (US); **Jonathan S. Parry**, Boise, ID (US); **Chulbum Kim**, San Jose, CA (US); **Daniel J. Hubbard**, Boise, ID (US); **Suresh Rajgopal**, San Diego, CA (US)

(73) Assignee: **Micron Technology, Inc.**, Boise, ID (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 249 days.

(21) Appl. No.: **18/137,002**

(22) Filed: **Apr. 20, 2023**

(65) **Prior Publication Data**

US 2023/0350587 A1 Nov. 2, 2023

Related U.S. Application Data

(60) Provisional application No. 63/337,300, filed on May 2, 2022.

(51) **Int. Cl.**
G06F 3/06 (2006.01)

(52) **U.S. Cl.**
CPC **G06F 3/0625** (2013.01); **G06F 3/0658** (2013.01); **G06F 3/0679** (2013.01)

(58) **Field of Classification Search**

CPC G06F 3/0625; G06F 3/0658; G06F 3/0679; G06F 3/0659; G06F 3/061

USPC 365/226
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

11,373,710 B1 *	6/2022	Hsu	G11C 16/30
2014/0195734 A1 *	7/2014	Ha	G06F 1/324
			711/114
2015/0235677 A1 *	8/2015	Grunzke	G11C 5/04
			365/226

* cited by examiner

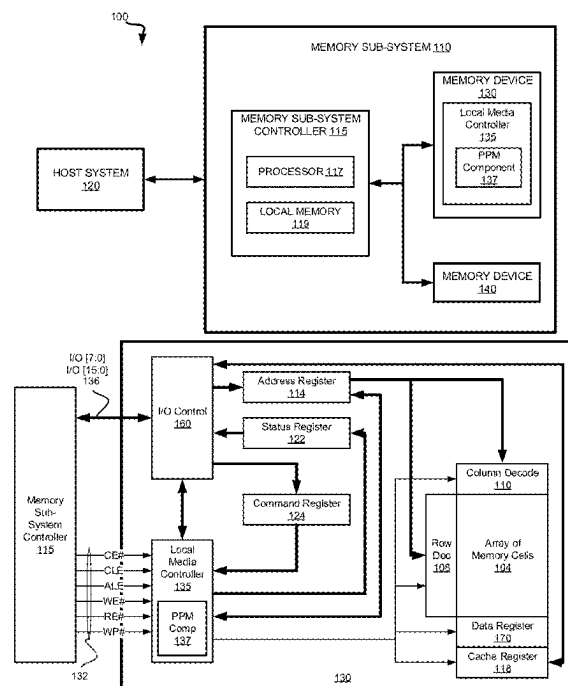
Primary Examiner — Fernando Hidalgo

(74) *Attorney, Agent, or Firm* — Lowenstein Sandler LLP

(57) **ABSTRACT**

A memory device includes memory dies, each memory die including a memory array and control logic, operatively coupled with the memory array, to perform peak power management (PPM) operations. The PPM operations include receiving a request to perform an operation, determining whether to initiate a PPM priority override procedure, and in response to determining to initiate the PPM priority override procedure, performing the PPM priority override procedure to execute the operation. Performing the PPM priority override procedure includes reconfiguring each high current breakpoints as a respective low current breakpoint to execute the operation.

20 Claims, 11 Drawing Sheets



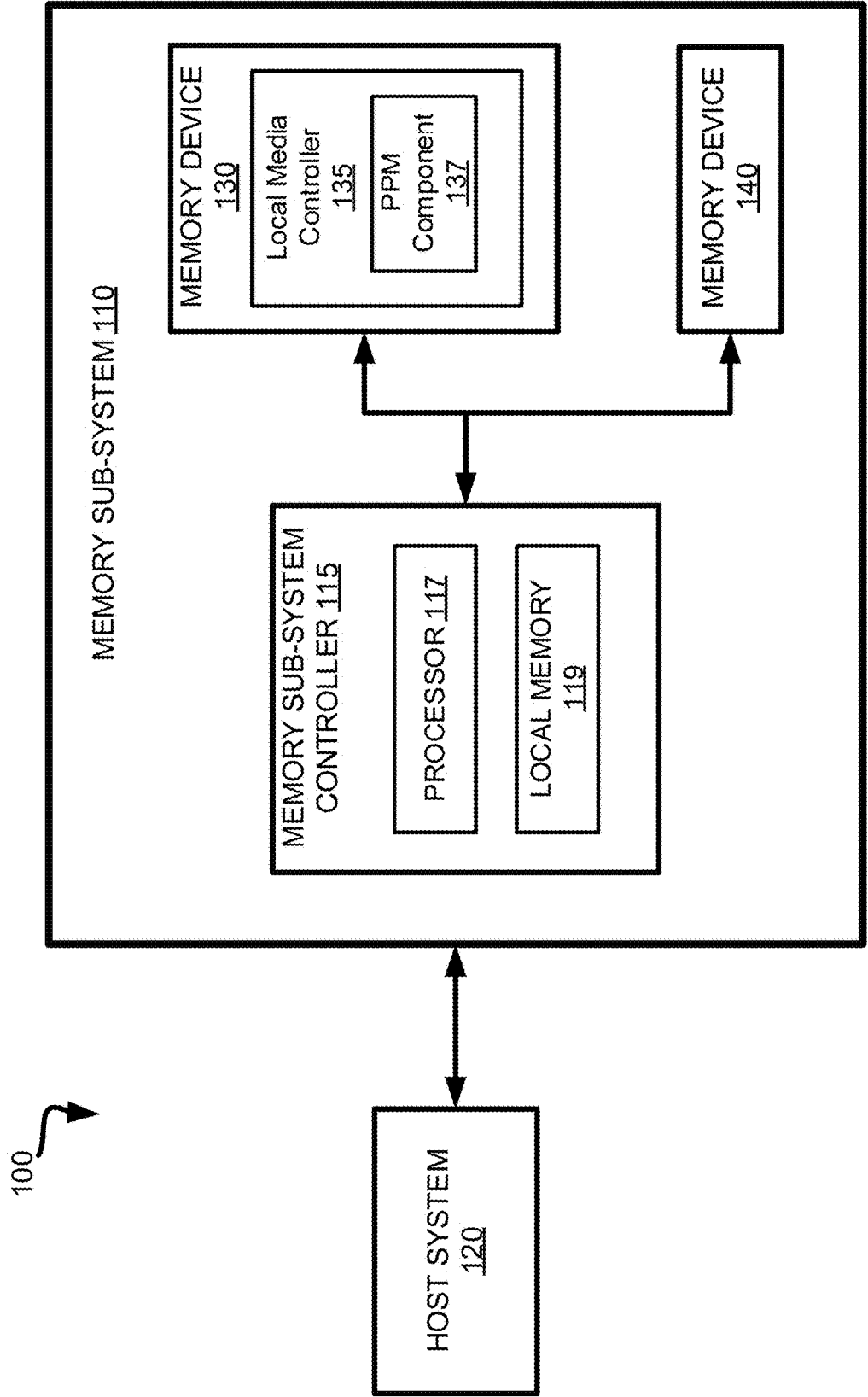


FIG. 1A

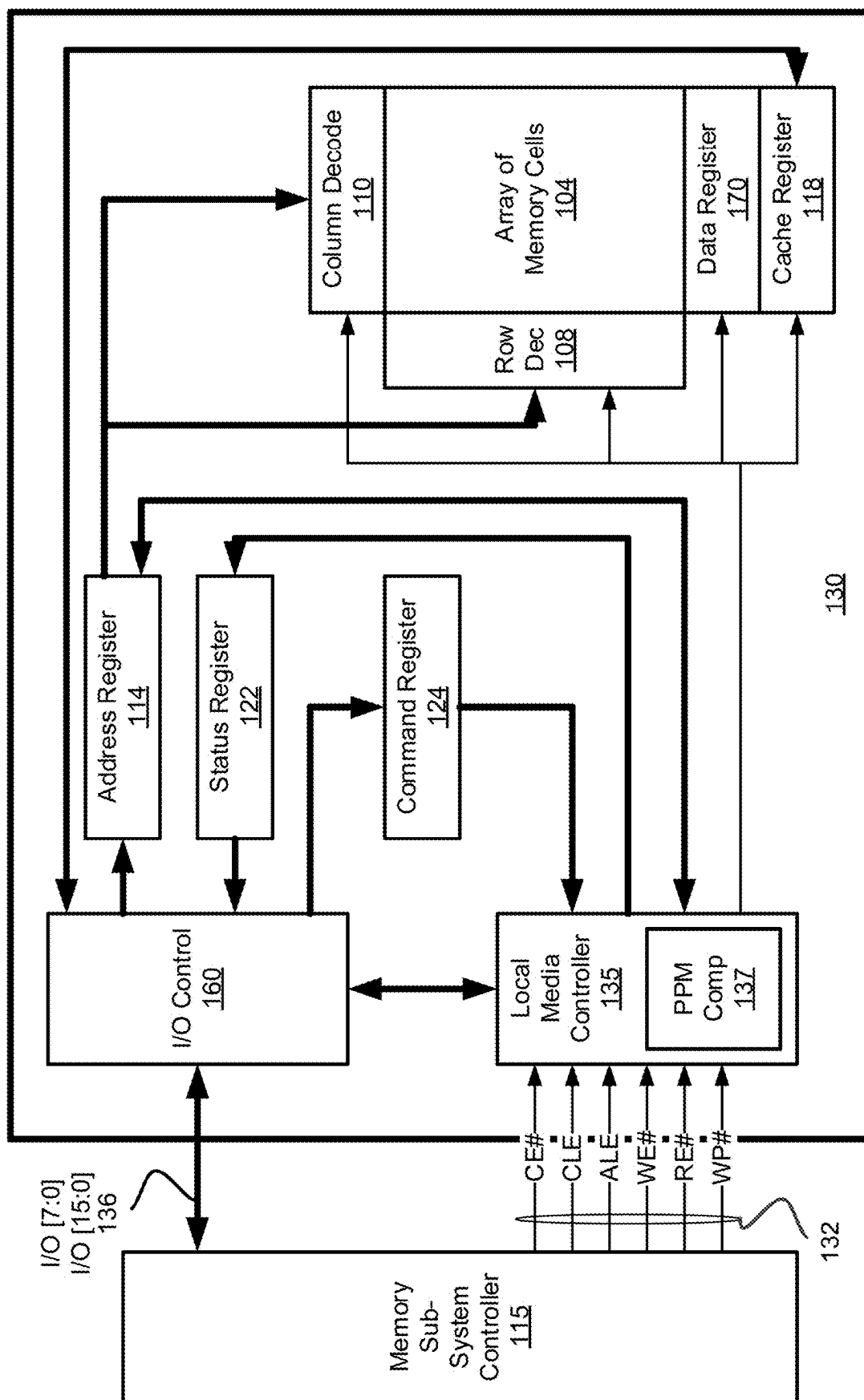


FIG. 1B

200A

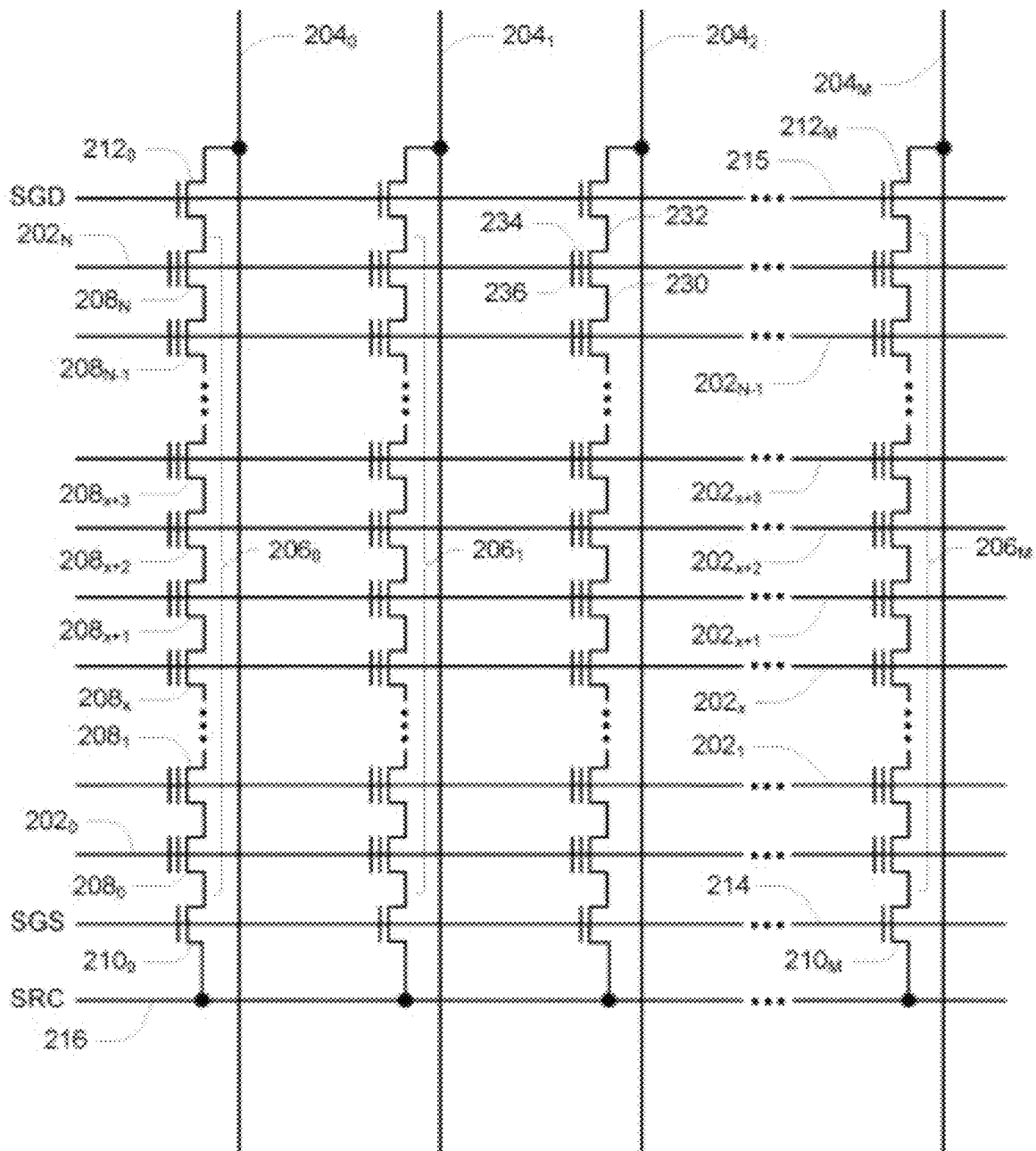


FIG. 2A

200B

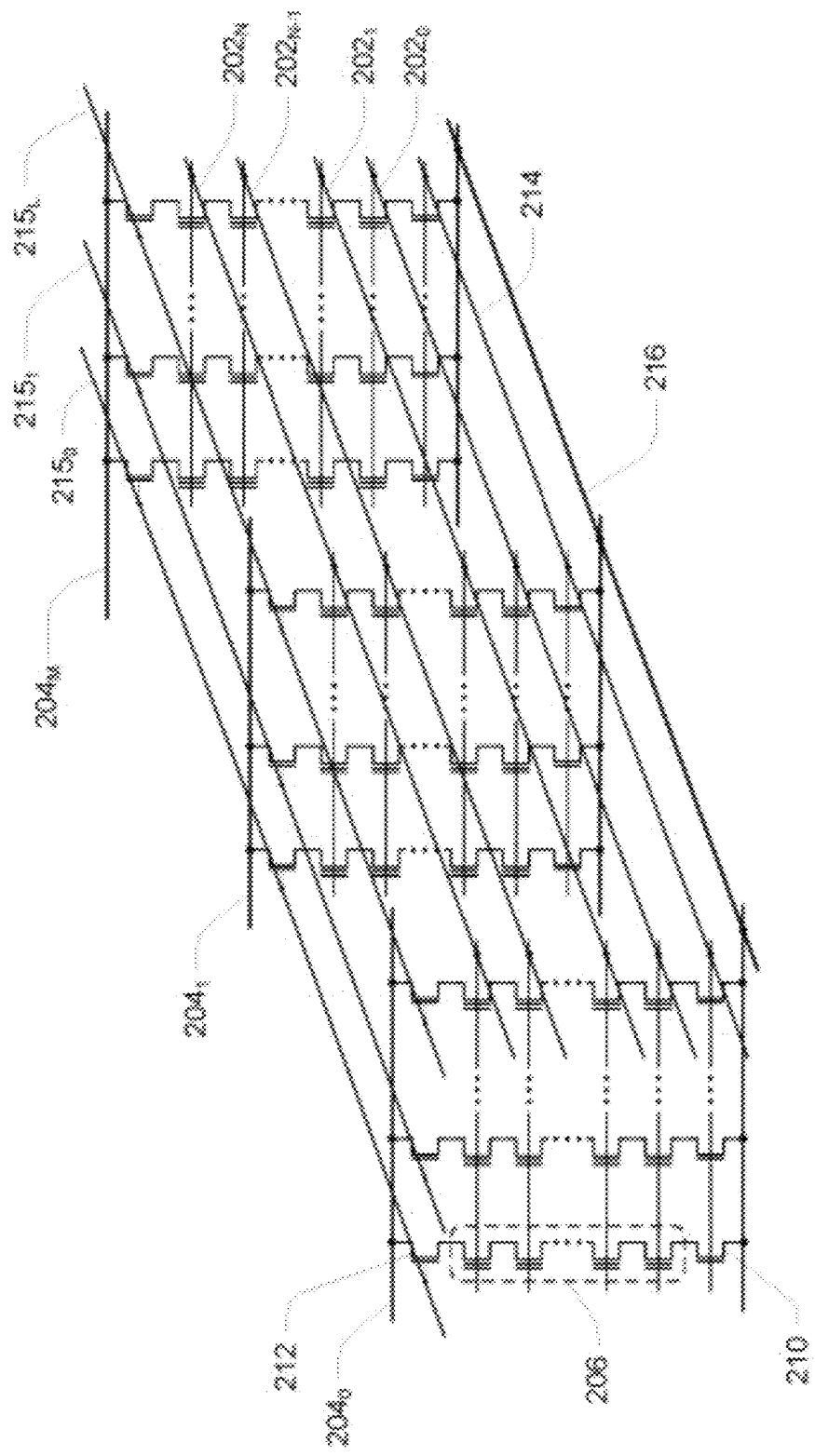


FIG. 2B

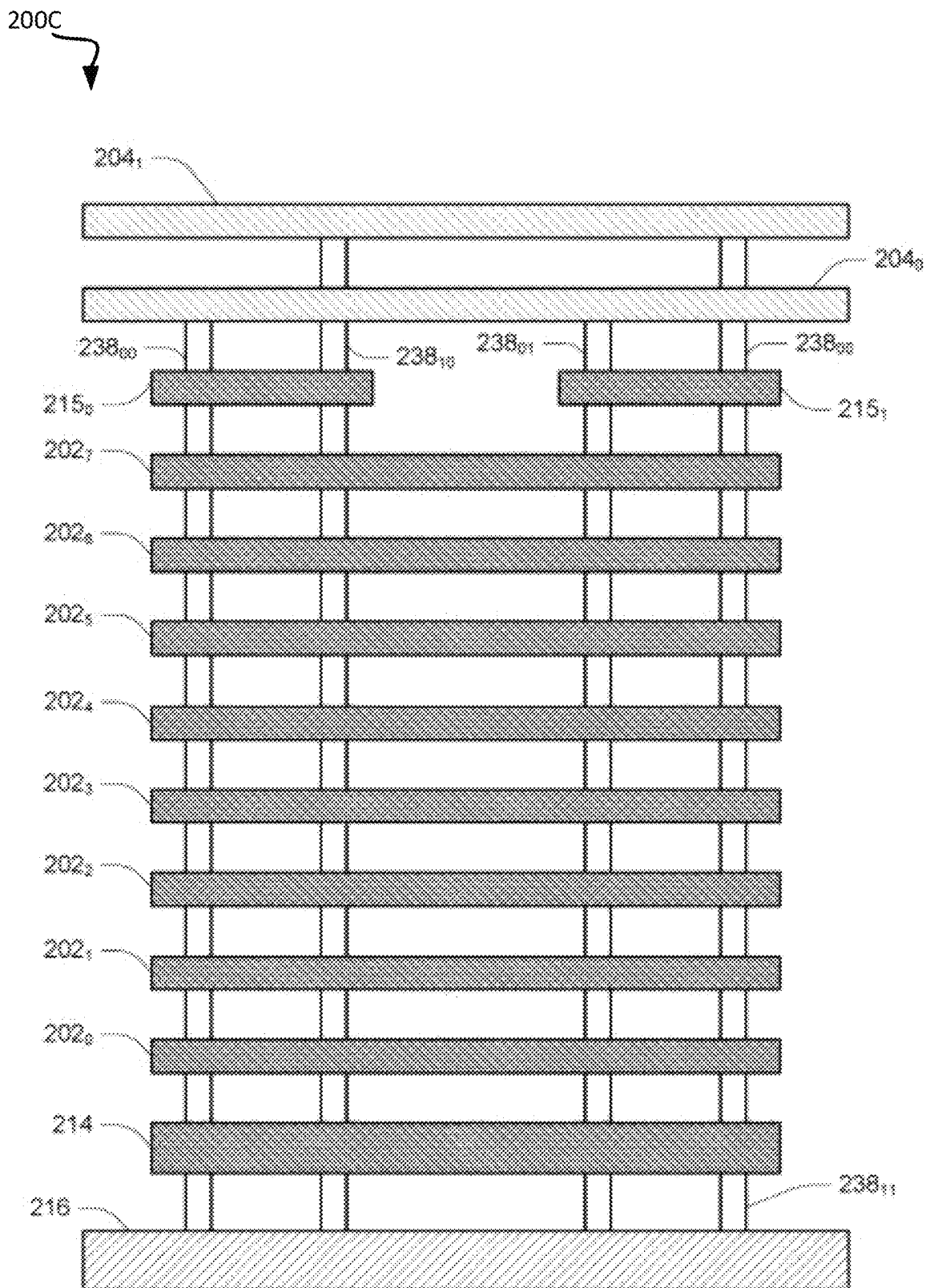


FIG. 2C

300

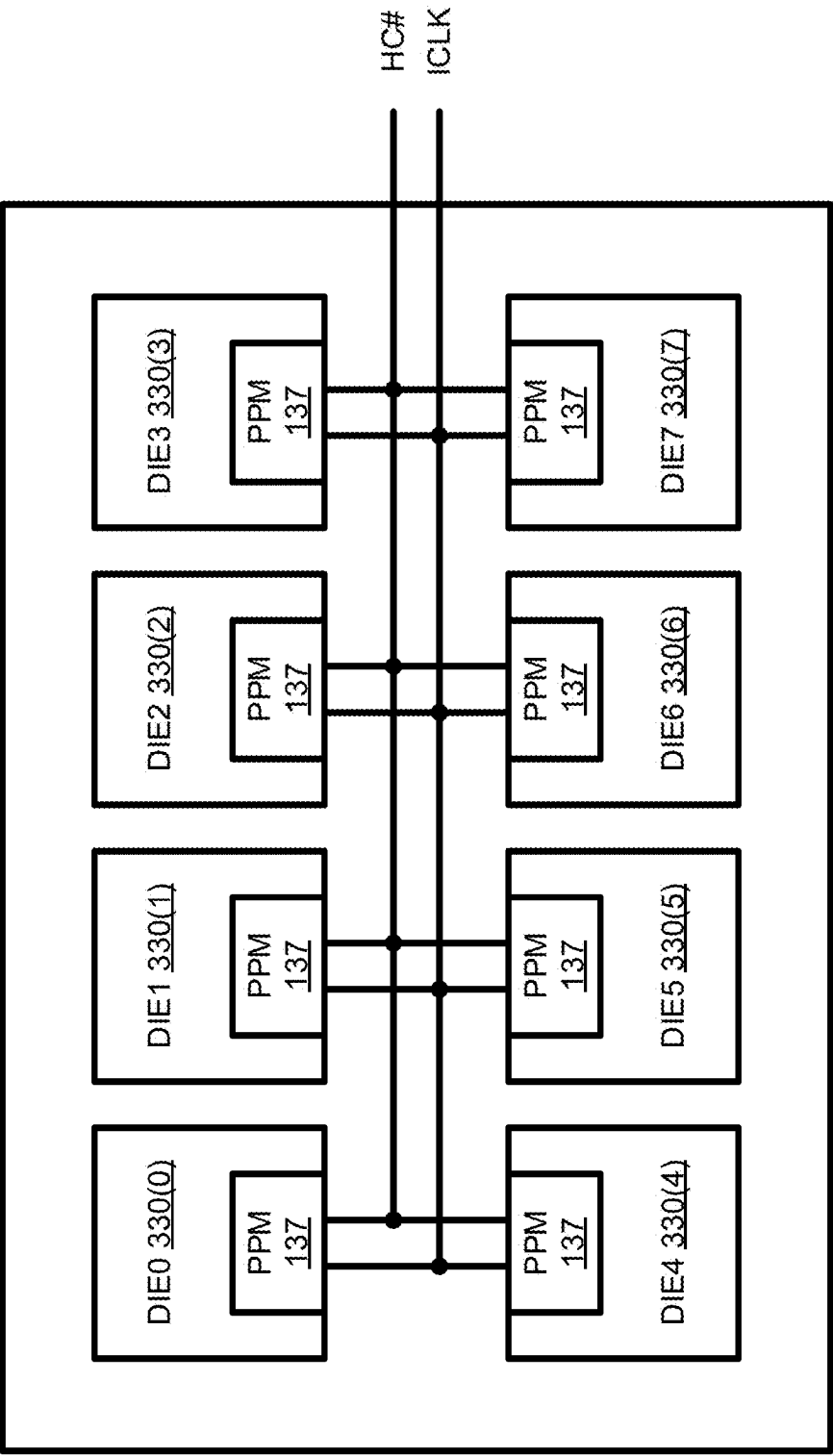


FIG. 3

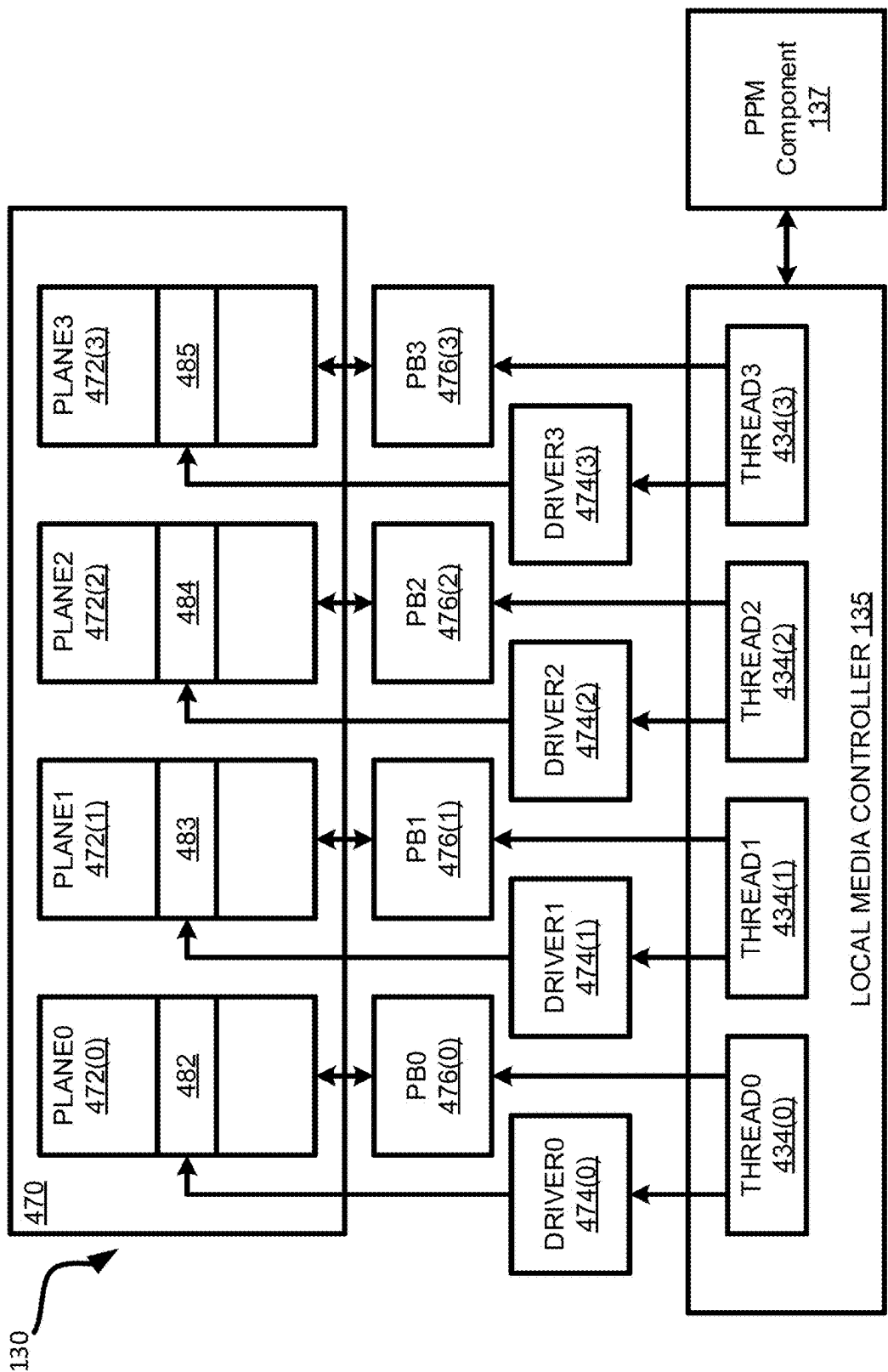
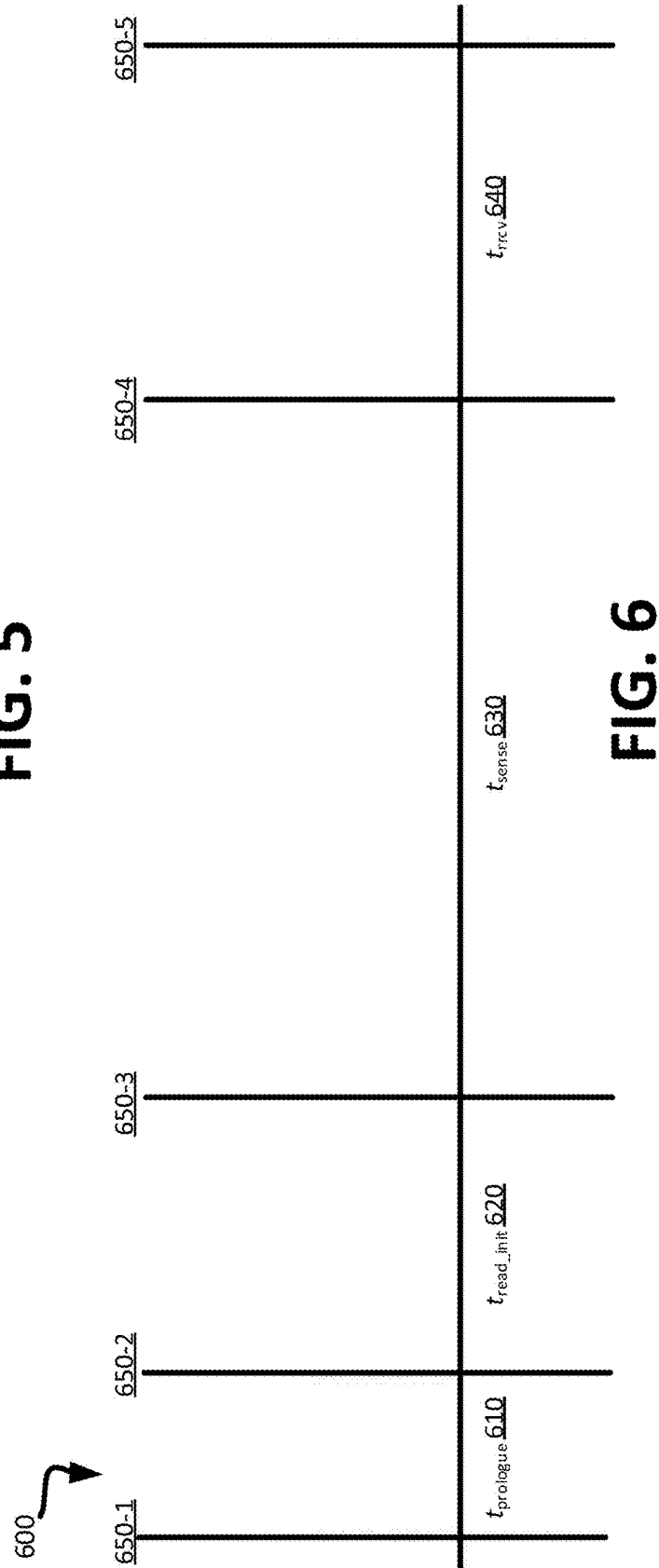
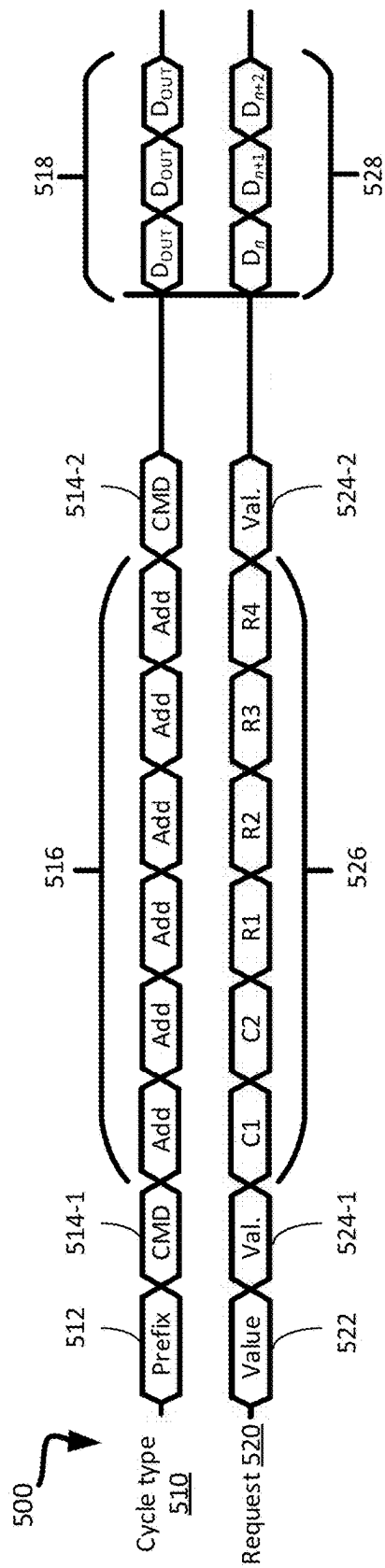


FIG. 4



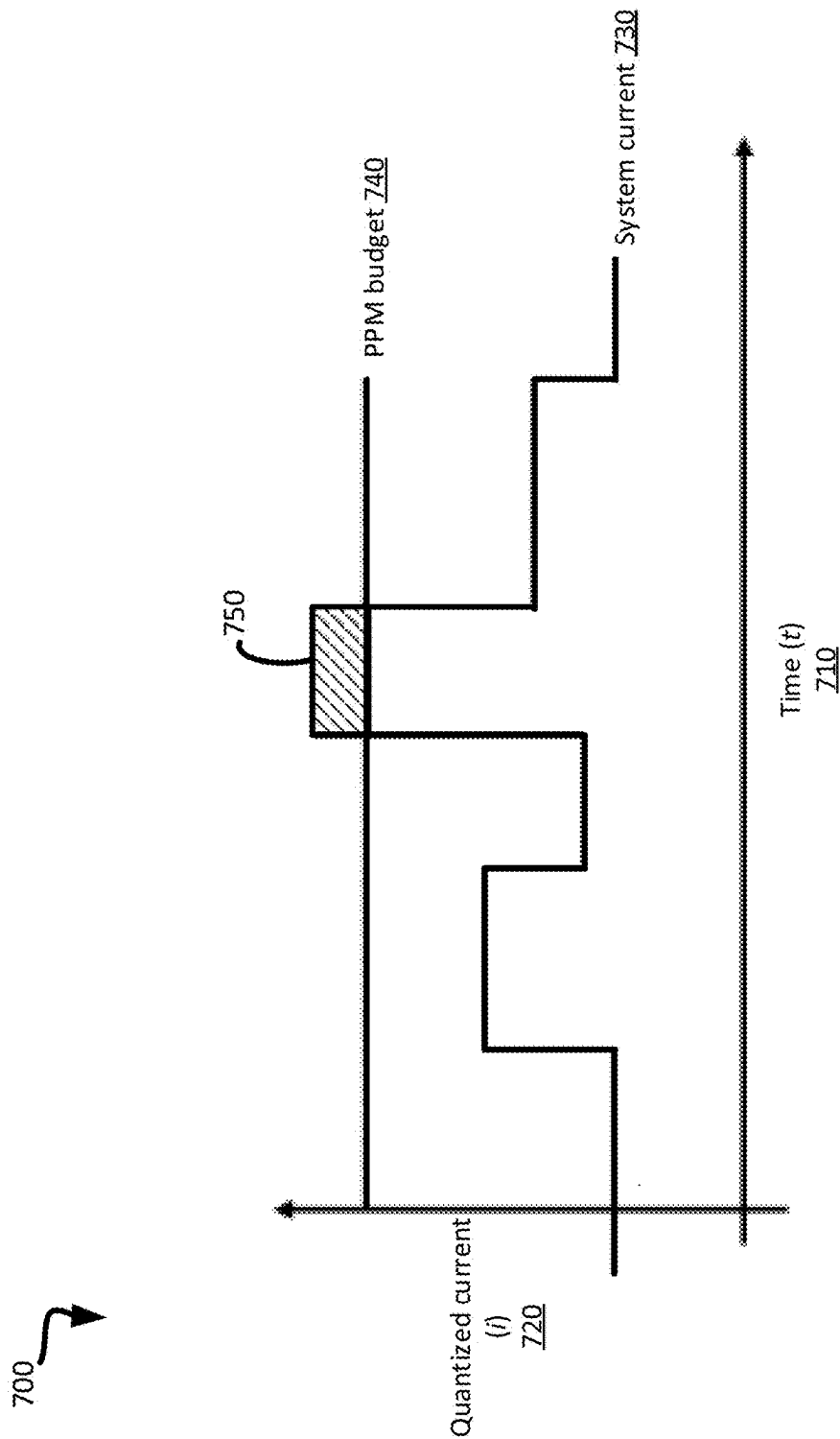


FIG. 7

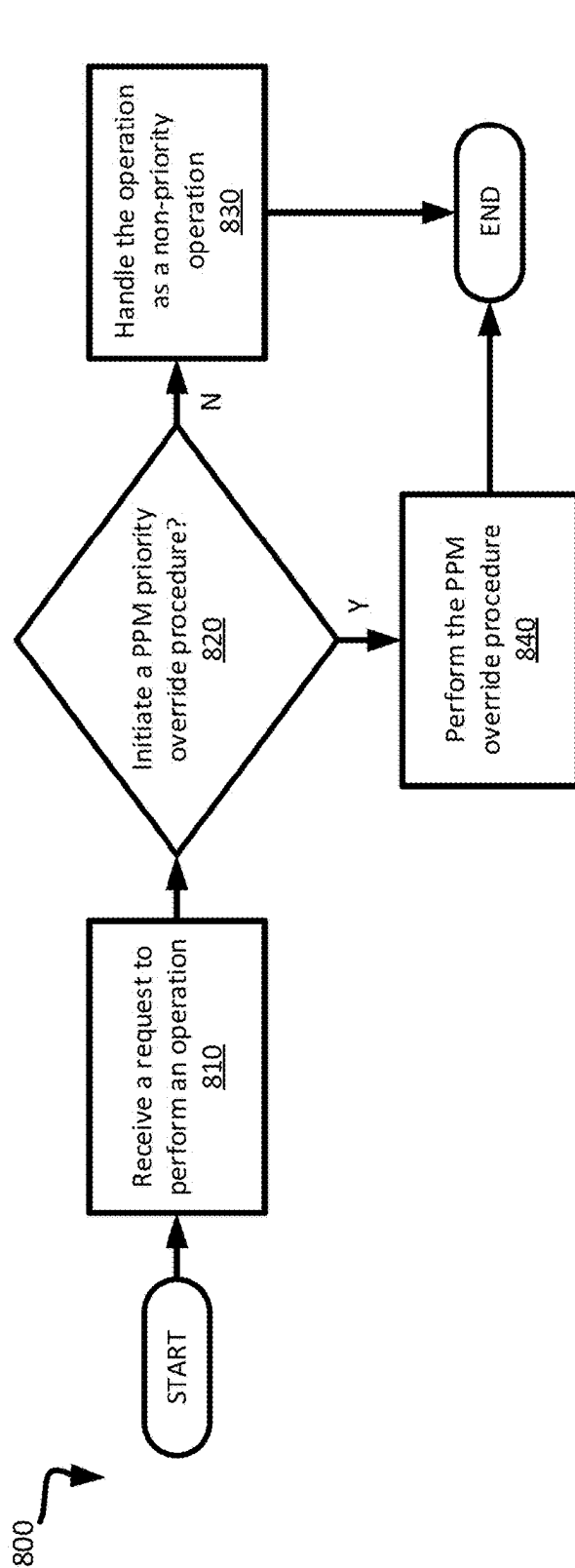


FIG. 8A

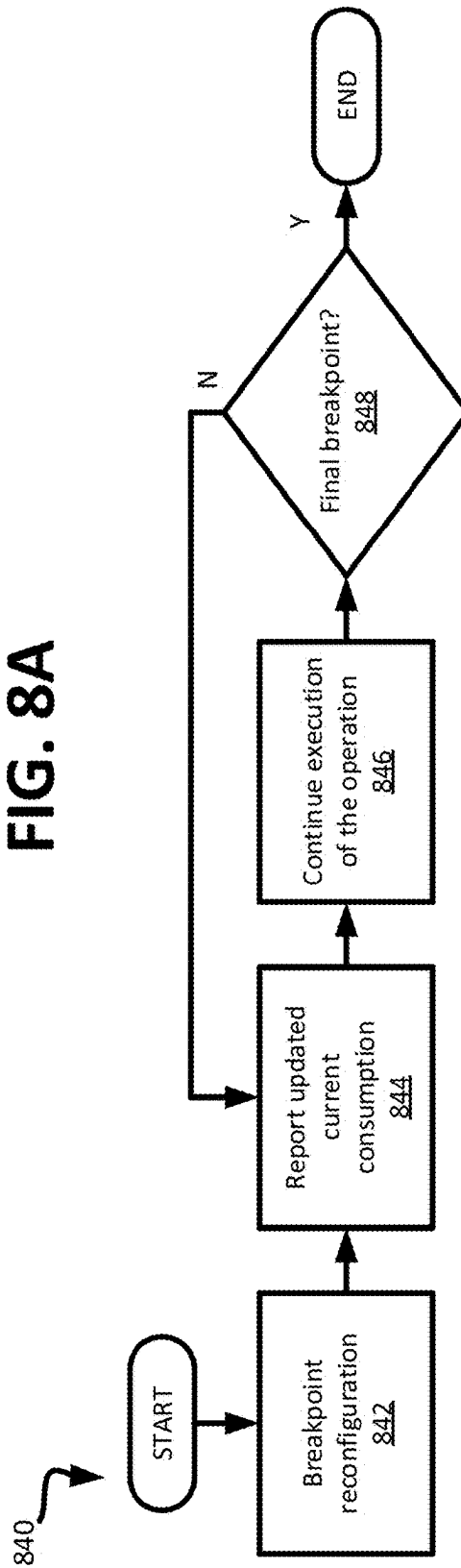
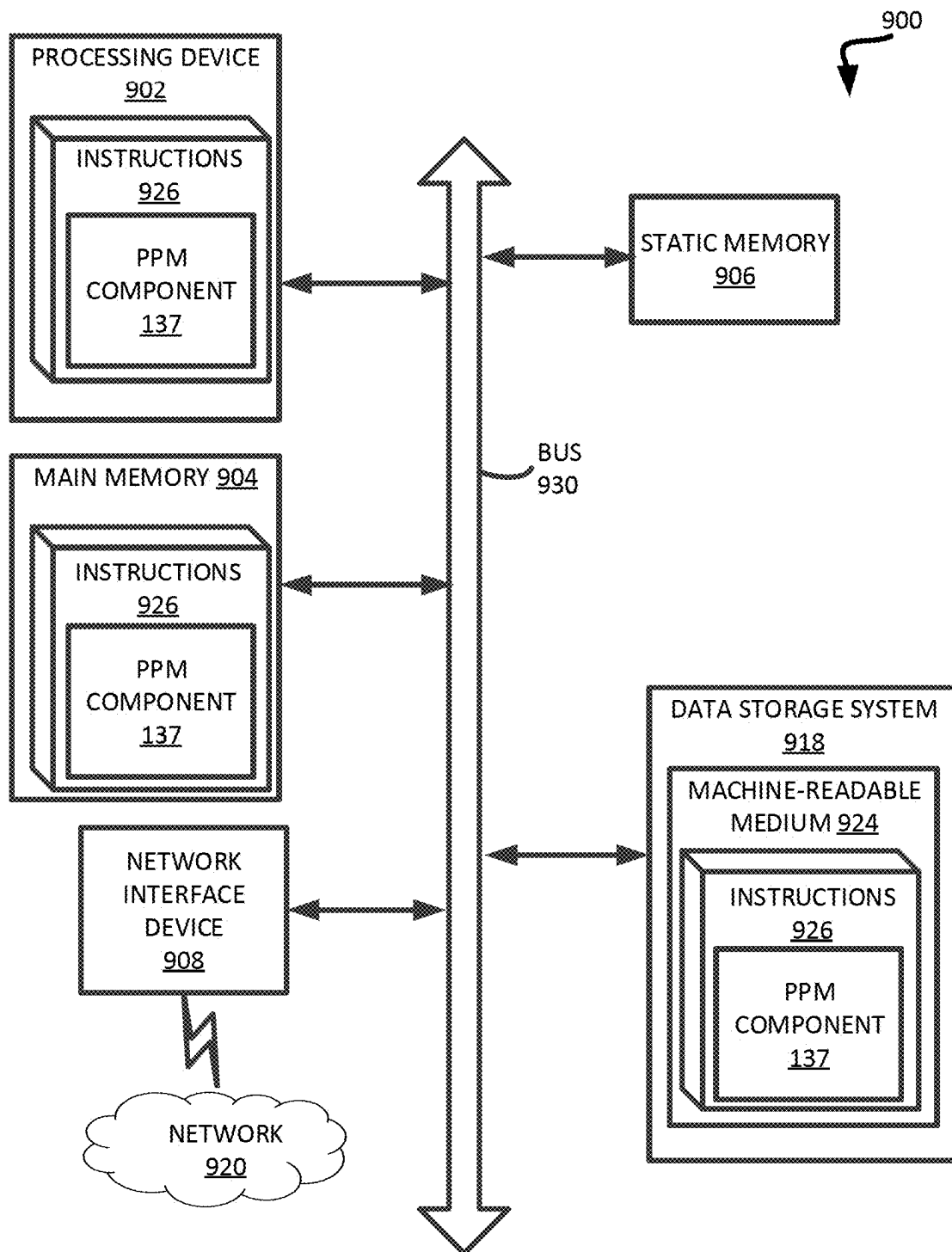


FIG. 8B

**FIG. 9**

1

**PEAK POWER MANAGEMENT PRIORITY
OVERRIDE**

RELATED APPLICATION

This application claims the benefit of U.S. Provisional Application 63/337,300 filed on May 2, 2022, the entire contents of which are incorporated by reference herein.

TECHNICAL FIELD

Embodiments of the disclosure relate generally to memory sub-systems, and more specifically, relate to priority override during peak power management in a memory device.

BACKGROUND

A memory sub-system can include one or more memory devices that store data. The memory devices can be, for example, non-volatile memory devices and volatile memory devices. In general, a host system can utilize a memory sub-system to store data at the memory devices and to retrieve data from the memory devices.

BRIEF DESCRIPTION OF THE DRAWINGS

The disclosure will be understood more fully from the detailed description given below and from the accompanying drawings of various embodiments of the disclosure. The drawings, however, should not be taken to limit the disclosure to the specific embodiments, but are for explanation and understanding only.

FIG. 1A illustrates an example computing system that includes a memory sub-system in accordance with some embodiments of the present disclosure.

FIG. 1B is a block diagram of a memory device in communication with a memory sub-system controller of a memory sub-system in accordance with some embodiments of the present disclosure.

FIGS. 2A-2C are diagrams of portions of an example array of memory cells included in a memory device, in accordance with some embodiments of the present disclosure.

FIG. 3 is a block diagram illustrating a multi-die package with multiple memory dies in a memory sub-system, in accordance with some embodiments of the present disclosure.

FIG. 4 is a block diagram illustrating a multi-plane memory device configured for independent parallel plane access, in accordance with some embodiments of the present disclosure.

FIG. 5 is a diagram of an example request to perform an operation, in accordance with some embodiments of the present disclosure.

FIG. 6 is a timing diagram of an example execution of a priority operation, in accordance with some embodiments of the present disclosure.

FIG. 7 is a graph illustrating current budget over time, in accordance with some embodiments of the present disclosure.

FIG. 8A is a flow diagram of a method to implement peak power management (PPM) in a memory device, in accordance with some embodiments of the present disclosure.

FIG. 8B is a flow diagram of a method to implement a peak power management (PPM) override procedure to execute an operation, in accordance with some embodiments of the present disclosure.

2

FIG. 9 is a block diagram of an example computer system in which embodiments of the present disclosure may operate.

DETAILED DESCRIPTION

Aspects of the present disclosure are directed to implementing priority override during peak power management (PPM). A memory sub-system can be a storage device, a memory module, or a combination of a storage device and memory module. Examples of storage devices and memory modules are described below in conjunction with FIGS. 1A-1B. In general, a host system can utilize a memory sub-system that includes one or more components, such as memory devices that store data. The host system can provide data to be stored at the memory sub-system and can request data to be retrieved from the memory sub-system.

A memory sub-system can include high density non-volatile memory devices where retention of data is desired when no power is supplied to the memory device. One example of non-volatile memory devices is a negative-and (NAND) memory device. Other examples of non-volatile memory devices are described below in conjunction with FIGS. 1A-1B. A non-volatile memory device is a package of one or more dies. Each die can consist of one or more planes. For some types of non-volatile memory devices (e.g., NAND devices), each plane consists of a set of physical blocks. Each block consists of a set of pages. Each page consists of a set of memory cells. A memory cell is an electronic circuit that stores information. Depending on the memory cell type, a memory cell can store one or more bits of binary information, and has various logic states that correlate to the number of bits being stored. The logic states can be represented by binary values, such as "0" and "1", or combinations of such values.

A memory device can include multiple bits arranged in a two-dimensional or three-dimensional grid. Memory cells are formed onto a silicon wafer in an array of columns (also hereinafter referred to as bitlines) and rows (also hereinafter referred to as wordlines). A wordline can refer to one or more rows of memory cells of a memory device that are used with one or more bitlines to generate the address of each of the memory cells. The intersection of a bitline and wordline constitutes the address of the memory cell. A block hereinafter refers to a unit of the memory device used to store data and can include a group of memory cells, a wordline group, a wordline, or individual memory cells. One or more blocks can be grouped together to form a plane of the memory device in order to allow concurrent operations to take place on each plane. The memory device can include circuitry that performs concurrent memory page accesses of two or more memory planes. For example, the memory device can include a respective access line driver circuit and power circuit for each plane of the memory device to facilitate concurrent access of pages of two or more memory planes, including different page types. For ease of description, these circuits can be generally referred to as independent plane driver circuits. Control logic on the memory device includes a number of separate processing threads to perform concurrent memory access operations (e.g., read operations, program operations, and erase operations). For example, each processing thread corresponds to a respective one of the memory planes and utilizes the associated independent plane driver circuits to perform the memory access operations on the respective memory plane. As these processing threads operate independently, the power usage and requirements associated with each processing thread also varies.

A memory device can be a three-dimensional (3D) memory device. For example, a 3D memory device can be a three-dimensional (3D) replacement gate memory device (e.g., 3D replacement gate NAND), which is a memory device with a replacement gate structure using wordline stacking. For example, a 3D replacement gate memory device can include wordlines, select gates, etc. located between sets of layers including a pillar (e.g., polysilicon pillar), a tunnel oxide layer, a charge trap (CT) layer, and a dielectric (e.g. oxide) layer. A 3D replacement gate memory device can have a “top deck” corresponding to a first side and a “bottom deck” corresponding to a second side. For example, the first side can be a drain side and the second side can be a source side. Data in a 3D replacement gate memory device can be stored as 1 bit/memory cell (SLC), 2 bits/memory cell (MLC), 3 bits/memory cell (TLC), etc.

The capacitive loading of 3D memory is generally large and may continue to grow as process scaling continues. Various access lines, data lines and voltage nodes can be charged or discharged very quickly during sense (e.g., read or verify), program, and erase operations so that memory array access operations can meet the performance specifications that are often required to satisfy data throughput targets as might be dictated by customer requirements or industry standards, for example. For sequential read or programming, multi-plane operations are often used to increase the system throughput. As a result, a typical memory device can have a high peak current usage, which might be four to five times the average current amplitude. Thus, with such a high average market requirement of total current usage budget, it can become challenging to concurrently operate more than a certain number of memory devices, such as four memory devices for example.

A variety of techniques have been utilized to manage power consumption of memory sub-systems containing multiple memory devices, many of which rely on a memory sub-system controller to stagger the activity of the memory devices seeking to avoid performing high power portions of access operations concurrently in more than one memory device. For example, in a memory package including multiple memory devices (e.g., multiple separate dies), there can be a peak power management (PPM) system. A PPM system implements a PPM communication protocol, which is an inter-memory-device (e.g. inter-die) communication protocol that can be used for limiting and/or tracking current or power consumed by the memory sub-system. Each memory device can include a PPM component (e.g., PPM manager) that exchanges information between its own local media controller (e.g., NAND controller) and the other PPM components of the PPM system via a communication bus. Each PPM component can be configured to perform power or current budget arbitration for the respective memory device. For example, each PPM component can implement predictive PPM to perform predictive power budget arbitration for the respective memory device.

The PPM communication protocol can employ a token-based round robin protocol, whereby each PPM component rotates as a holder of the token in accordance with a token circulation time period. Circulation of the token among the memory devices can be controlled by a common clock signal (“ICLK”). For example, the memory devices (e.g., dies) can include a designated primary memory device that generates the common clock signal received by each active PPM component. The token circulation time period can be defined by a number of clock cycles of the common clock signal, and the memory device can pass the token to the next memory device after the number of clock cycles has elapsed.

For example, the token circulation time period can be defined by three clock cycles.

A memory device counter (e.g., die counter) can be used by each memory device to keep track of which memory device is holding the token. More specifically, the memory device counter can assign a memory device value to indicate the memory device that is currently holding the token. Each memory device counter value can be univocally associated with a respective memory device by utilizing a special PPM address for each memory device. The memory device counter can be updated upon the passing of the token to the next memory device.

While holding the token, the PPM component broadcasts, to the other memory devices, information codifying the amount of current used by its respective memory device during a given time period (e.g., a quantized current budget). The information can be broadcast using a data line. For example, the data line can be a high current (HC #) data line. The amount of information can be defined by a number of bits, where each bit corresponds to the logic level of a data line signal (e.g., an HC # signal) at a respective clock cycle (e.g., a bit has a value of “0” if the HC # signal is logic low during a clock cycle, or a value of “1” if the clock pulse is logic high during a clock cycle). For example, if a memory device circulates the token after three clock cycles, then the information can include three bits of information. More specifically, a first bit corresponds to the logic level of the HC # signal during a first clock cycle, a second bit corresponds to the logic level of the HC # signal during a second clock cycle, and a third bit corresponds to the logic level of the HC # signal during the third clock cycle. Accordingly, the token circulation time period (e.g., number of clock cycles) can be defined in accordance with the amount of information to be broadcast by a holder of the token (e.g., number of bits).

While holding the token, the PPM component can issue a new request for a certain amount of current for its respective memory device to consume to execute an operation. The system can have a designated maximum current budget, and at least a portion of the maximum current budget may be consumed by other memory devices during execution of previous operations (“current consumption”). Thus, an available current budget can be defined as the difference between the maximum current budget and the current consumption. If the amount of current of the new request is less than or equal to the available current budget, then the request is granted and the local media controller can cause the operation to be executed. Otherwise, if the amount of current of the new request exceeds the available current budget, then the local media controller can be forced to wait for enough current budget to be made available to execute the operation.

Each PPM component can maintain the information broadcast by each memory device within respective registers, which enables each memory device to calculate the current consumption. For example, if there are four dies Die 0 through Die 3, each Die 0 through Die 3 can maintain information broadcast by Die 0 through Die 3 within respective registers designated for Die 0 through Die 3. Since each memory device maintains the maximum current budget the most updated current consumption, each memory device can calculate the available current budget. Accordingly, each memory device can determine whether there is a sufficient amount of available current budget for its local media controller to execute a new operation.

For example, high current (HC) breakpoints can be used as a gating mechanism to ensure that sufficient current is available before allowing a local media controller of a

5

memory device to proceed with execution of a sub-operation. A HC breakpoint indicates a point during performance of an operation at which the memory device (e.g., die) will be consuming a greater amount of current greater than it was just consuming (e.g., more current is being reserved than before). For example, an operation can include a number of sub-operations. A first HC breakpoint can be placed before execution of an initial sub-operation, since the initial sub-operation will cause the memory device to consume a greater amount of current than the zero amount of current that was being consumed immediately before requesting execution of the operation. Therefore, an HC breakpoint can be placed before execution of a sub-operation that will consume more current than the previous sub-operation. Illustratively, if the operation is a read operation, the first HC breakpoint can serve as a gate to initiate the performance of the initial prologue sub-operation, and a second HC breakpoint can serve as a gate to initiate the performance of the a read initialization sub-operation following the prologue sub-operation (since the read initialization sub-operation consumes more current than the prologue sub-operation). Upon reaching a HC breakpoint, the local media controller can communicate, with the PPM component, the amount of current that the memory device will be consuming to perform the operation. The local media controller waits to receive a response (e.g., flag) indicating that there is sufficient available current budget to reserve for performing the operation. Upon receiving the response from that PPM component that there is sufficient available current budget, the local media controller can proceed with performing the operation.

The remaining sub-operations (e.g., sensing sub-operation and read recovery sub-operation of a read operation) can be separated by low current (LC) breakpoints. In contrast to a HC breakpoint, a LC breakpoint indicates a point during the performance of the operation at which the memory device will be consuming an amount of current that is less than or equal to the previous sub-operation (e.g., at most the same amount of current will be reserved as compared to before). Therefore, since the memory device does not need to reserve additional current to proceed, the local media controller will proceed and not wait for the PPM component to provide a response upon reaching a LC breakpoint. However, the local media controller still communicates, with the PPM component, the amount of current that the memory device will be consuming to perform the sub-operation following a LC breakpoint.

Mixed workload systems enabling PPM (e.g., predictive PPM) may have a need to execute priority operations, such as a priority read operation (e.g., snap read or full page read). A priority operation can refer to any operation that must execute, regardless of the available amount of current budget. For example, a priority read operation can be received while the multiple memory devices are currently executing write operations. However, typical ways of handling priority operations within a memory sub-system implementing PPM can introduce performance penalties that negatively affect quality of service (QoS). For example, non-priority operations that are currently being executed by the memory devices may need to be suspended and/or paused to free up available current budget. Suspending and/or pausing non-priority operations can introduce memory device overhead.

Aspects of the present disclosure address the above and other deficiencies by implementing peak power management (PPM) priority override for a memory device (e.g., die) of a memory sub-system to perform a priority operation, without first causing other memory devices of the memory

6

sub-system to suspend and/or pause current operations to increase the amount of available current budget. For example, a local media controller of a memory device can receive a request from a host (e.g., from the memory sub-system controller) to perform an operation. The local media controller can determine whether the operation is a priority operation triggering PPM priority override. In some embodiments, the request includes a prefix command. For example, the local media controller can determine whether the request includes a prefix indicating that a command to perform an operation is a priority command to perform a priority operation triggering PPM priority override. The prefix is located before the command sequence. In some embodiments, the request includes a set feature command. For example, the local media controller can determine whether the request includes a set feature. In some embodiments, the PPM priority override can be automatically enabled when PPM is active.

If the operation is determined to be a priority operation, then the local media controller can reconfigure the HC breakpoints as LC breakpoints (e.g., in the case of a read operation, the HC breakpoints that define the boundaries of the read prologue sub-operation). The local media controller can then execute the priority operation without determining if there is sufficient available current budget to execute the priority operation. For example, the local media controller can communicate a new current value to the PPM component and then continue executing the priority operation without delay. The PPM component can report the amount of current being consumed to the other memory devices (e.g., other dies). Thus, when the other memory devices reach their HC breakpoints, the other memory devices can account for the reported amount of current with respect to the remaining current budget. This can cause the other memory devices to pause or suspend their operations until enough current is made available. In instances where the PPM priority override causes the current budget to be exceeded, the memory sub-system can manage the number of overrides active at any given time (e.g., using a credit-based system).

Advantages of the present disclosure include, but are not limited to, improved memory sub-system performance and QoS. For example, by enabling a memory device of a memory sub-system implementing PPM (e.g., die) to execute a priority operation without waiting for sufficient current budget or having to suspend and/or pause other operations being performed by other memory devices of the memory sub-system (e.g., other dies), memory sub-system overhead and/or additional performance penalties can be reduced.

FIG. 1A illustrates an example computing system **100** that includes a memory sub-system **110** in accordance with some embodiments of the present disclosure. The memory sub-system **110** can include media, such as one or more volatile memory devices (e.g., memory device **140**), one or more non-volatile memory devices (e.g., memory device **130**), or a combination of such.

A memory sub-system **110** can be a storage device, a memory module, or a combination of a storage device and memory module. Examples of a storage device include a solid-state drive (SSD), a flash drive, a universal serial bus (USB) flash drive, an embedded Multi-Media Controller (eMMC) drive, a Universal Flash Storage (UFS) drive, a secure digital (SD) card, and a hard disk drive (HDD). Examples of memory modules include a dual in-line memory module (DIMM), a small outline DIMM (SO-

DIMM), and various types of non-volatile dual in-line memory modules (NVDIMMs).

The computing system **100** can be a computing device such as a desktop computer, laptop computer, network server, mobile device, a vehicle (e.g., airplane, drone, train, automobile, or other conveyance), Internet of Things (IoT) enabled device, embedded computer (e.g., one included in a vehicle, industrial equipment, or a networked commercial device), or such computing device that includes memory and a processing device.

The computing system **100** can include a host system **120** that is coupled to one or more memory sub-systems **110**. In some embodiments, the host system **120** is coupled to multiple memory sub-systems **110** of different types. FIG. 1A illustrates one example of a host system **120** coupled to one memory sub-system **110**. As used herein, “coupled to” or “coupled with” generally refers to a connection between components, which can be an indirect communicative connection or direct communicative connection (e.g., without intervening components), whether wired or wireless, including connections such as electrical, optical, magnetic, etc.

The host system **120** can include a processor chipset and a software stack executed by the processor chipset. The processor chipset can include one or more cores, one or more caches, a memory controller (e.g., NVDIMM controller), and a storage protocol controller (e.g., PCIe controller, SATA controller). The host system **120** uses the memory sub-system **110**, for example, to write data to the memory sub-system **110** and read data from the memory sub-system **110**.

The host system **120** can be coupled to the memory sub-system **110** via a physical host interface. Examples of a physical host interface include, but are not limited to, a serial advanced technology attachment (SATA) interface, a peripheral component interconnect express (PCIe) interface, universal serial bus (USB) interface, Fibre Channel, Serial Attached SCSI (SAS), a double data rate (DDR) memory bus, Small Computer System Interface (SCSI), a dual in-line memory module (DIMM) interface (e.g., DIMM socket interface that supports Double Data Rate (DDR)), etc. The physical host interface can be used to transmit data between the host system **120** and the memory sub-system **110**. The host system **120** can further utilize an NVM Express (NVMe) interface to access components (e.g., memory devices **130**) when the memory sub-system **110** is coupled with the host system **120** by the physical host interface (e.g., PCIe bus). The physical host interface can provide an interface for passing control, address, data, and other signals between the memory sub-system **110** and the host system **120**. FIG. 1A illustrates a memory sub-system **110** as an example. In general, the host system **120** can access multiple memory sub-systems via a same communication connection, multiple separate communication connections, and/or a combination of communication connections.

The memory devices **130**, **140** can include any combination of the different types of non-volatile memory devices and/or volatile memory devices. The volatile memory devices (e.g., memory device **140**) can be, but are not limited to, random access memory (RAM), such as dynamic random access memory (DRAM) and synchronous dynamic random access memory (SDRAM).

Some examples of non-volatile memory devices (e.g., memory device **130**) include a negative-and (NAND) type flash memory and write-in-place memory, such as a three-dimensional cross-point (“3D cross-point”) memory device, which is a cross-point array of non-volatile memory cells. A cross-point array of non-volatile memory cells can perform

bit storage based on a change of bulk resistance, in conjunction with a stackable cross-gridded data access array. Additionally, in contrast to many flash-based memories, cross-point non-volatile memory can perform a write in-place operation, where a non-volatile memory cell can be programmed without the non-volatile memory cell being previously erased. NAND type flash memory includes, for example, two-dimensional NAND (2D NAND) and three-dimensional NAND (3D NAND).

Each of the memory devices **130** can include one or more arrays of memory cells. One type of memory cell, for example, single level memory cells (SLC) can store one bit per memory cell. Other types of memory cells, such as multi-level memory cells (MLCs), triple level memory cells (TLCs), quad-level memory cells (QLCs), and penta-level memory cells (PLCs) can store multiple bits per memory cell. In some embodiments, each of the memory devices **130** can include one or more arrays of memory cells such as SLCs, MLCs, TLCs, QLCs, PLCs or any combination of such. In some embodiments, a particular memory device can include an SLC portion, and an MLC portion, a TLC portion, a QLC portion, or a PLC portion of memory cells. The memory cells of the memory devices **130** can be grouped as pages that can refer to a logical unit of the memory device used to store data. With some types of memory (e.g., NAND), pages can be grouped to form blocks.

Although non-volatile memory components such as a 3D cross-point array of non-volatile memory cells and NAND type flash memory (e.g., 2D NAND, 3D NAND) are described, the memory device **130** can be based on any other type of non-volatile memory, such as read-only memory (ROM), phase change memory (PCM), self-selecting memory, other chalcogenide based memories, ferroelectric transistor random-access memory (FeTRAM), ferroelectric random access memory (FeRAM), magneto random access memory (MRAM), Spin Transfer Torque (STT)-MRAM, conductive bridging RAM (CBRAM), resistive random access memory (RRAM), oxide based RRAM (OxRAM), negative-or (NOR) flash memory, or electrically erasable programmable read-only memory (EEPROM).

A memory sub-system controller **115** (or controller **115** for simplicity) can communicate with the memory devices **130** to perform operations such as reading data, writing data, or erasing data at the memory devices **130** and other such operations. The memory sub-system controller **115** can include hardware such as one or more integrated circuits and/or discrete components, a buffer memory, or a combination thereof. The hardware can include a digital circuitry with dedicated (i.e., hard-coded) logic to perform the operations described herein. The memory sub-system controller **115** can be a microcontroller, special purpose logic circuitry (e.g., a field programmable gate array (FPGA), an application specific integrated circuit (ASIC), etc.), or other suitable processor.

The memory sub-system controller **115** can include a processing device, which includes one or more processors (e.g., processor **117**), configured to execute instructions stored in a local memory **119**. In the illustrated example, the local memory **119** of the memory sub-system controller **115** includes an embedded memory configured to store instructions for performing various processes, operations, logic flows, and routines that control operation of the memory sub-system **110**, including handling communications between the memory sub-system **110** and the host system **120**.

In some embodiments, the local memory **119** can include memory registers storing memory pointers, fetched data, etc.

The local memory **119** can also include read-only memory (ROM) for storing micro-code. While the example memory sub-system **110** in FIG. 1A has been illustrated as including the memory sub-system controller **115**, in another embodiment of the present disclosure, a memory sub-system **110** does not include a memory sub-system controller **115**, and can instead rely upon external control (e.g., provided by an external host, or by a processor or controller separate from the memory sub-system).

In general, the memory sub-system controller **115** can receive commands or operations from the host system **120** and can convert the commands or operations into instructions or appropriate commands to achieve the desired access to the memory devices **130**. The memory sub-system controller **115** can be responsible for other operations such as wear leveling operations, garbage collection operations, error detection and error-correcting code (ECC) operations, encryption operations, caching operations, and address translations between a logical address (e.g., a logical block address (LBA), namespace) and a physical address (e.g., physical block address) that are associated with the memory devices **130**. The memory sub-system controller **115** can further include host interface circuitry to communicate with the host system **120** via the physical host interface. The host interface circuitry can convert the commands received from the host system into command instructions to access the memory devices **130** as well as convert responses associated with the memory devices **130** into information for the host system **120**.

The memory sub-system **110** can also include additional circuitry or components that are not illustrated. In some embodiments, the memory sub-system **110** can include a cache or buffer (e.g., DRAM) and address circuitry (e.g., a row decoder and a column decoder) that can receive an address from the memory sub-system controller **115** and decode the address to access the memory devices **130**.

In some embodiments, the memory devices **130** include local media controllers **135** that operate in conjunction with memory sub-system controller **115** to execute operations on one or more memory cells of the memory devices **130**. An external controller (e.g., memory sub-system controller **115**) can externally manage the memory device **130** (e.g., perform media management operations on the memory device **130**). In some embodiments, memory sub-system **110** is a managed memory device, which is a raw memory device **130** having control logic (e.g., local media controller **135**) on the die and a controller (e.g., memory sub-system controller **115**) for media management within the same memory device package. An example of a managed memory device is a managed NAND (MNAND) device.

The local media controller **135** can implement a peak power management (PPM) component **137** that can perform PPM override in the memory device **130**. In such an embodiment, PPM component **137** can be implemented using hardware or as firmware, stored on memory device **130**, executed by the control logic (e.g., local media controller **135**) to perform the operations related to performing PPM during a program/erase suspend status as described herein. In some embodiments, the memory sub-system controller **115** includes at least a portion of PPM component **137**. For example, the memory sub-system controller **115** can include a processor **117** (e.g., a processing device) configured to execute instructions stored in local memory **119** for performing the operations described herein.

For example, the PPM component **137** can be included within a memory die ("die") of a multi-die memory device. For example, memory device **130** can represent one memory

die and can include PPM component **137** as illustrated. Similarly, memory sub-system **110** can include multiple other memory devices (i.e., separate memory dies), which can each include a respective PPM component. Upon receiving a request from a host (e.g., from the memory sub-system controller **115**) to perform an operation, the local media controller **135** can determine whether the operation is a priority operation triggering PPM priority override.

In some embodiments, the request includes a prefix command. For example, the local media controller **135** can determine whether the request includes a prefix indicating that a command to perform an operation is a priority command to perform a priority operation triggering PPM priority override. The prefix is located before the command sequence. In some embodiments, the request includes a set feature command. For example, the local media controller **135** can determine whether the request includes a set feature. In some embodiments, the PPM priority override can be automatically enabled when PPM is active.

In response to determining that the operation is a priority operation, the local media controller **135** can reconfigure high current (HC) breakpoints as low current (LC) breakpoints. For example, one HC breakpoint can serve as a gate to initiate the performance of an initial sub-operation of the operation (e.g., in the case of a read operation, before executing a prologue sub-operation of a read operation), and another HC breakpoint can serve as a gate to initiate the performance of a subsequent sub-operation of the operation (e.g., in the case of a read operation, before executing a read initialization sub-operation after completion of the prologue sub-operation). Reconfiguring the HC breakpoints as LC breakpoints enables the local media controller **135** to proceed with execution of the priority operation without having to wait for the PPM component **137** to confirm that there is sufficient available current budget to perform the priority operation.

The local media controller **135** can then execute the priority operation without determining if there is sufficient available current budget to execute the priority operation. For example, the local media controller **135** can communicate a new current value to the PPM component **137** and then continue executing the priority operation without delay. The PPM component **137** can report the amount of current being consumed to the other memory devices (e.g., other dies). Thus, when other memory devices reach their HC breakpoints, the other memory devices can account for the reported amount of current with respect to the remaining current budget. This can cause the other memory devices to pause or suspend their operations until enough current is made available. In instances where the PPM priority override causes the current budget to be exceeded, the memory sub-system can manage the number of overrides active at any given time (e.g., using a credit-based system). Accordingly, embodiments described herein can allow priority operations to execute in a PPM (e.g., predictive PPM) environment without additional performance penalty. Further details regarding the operations of the local media controller **135** and PPM component **137** are described above, and will be described below with reference to FIGS. 5-8.

FIG. 1B is a simplified block diagram of a first apparatus, in the form of a memory device **130**, in communication with a second apparatus, in the form of a memory sub-system controller **115** of a memory sub-system (e.g., memory sub-system **110** of FIG. 1A), according to an embodiment. Some examples of electronic systems include personal computers, personal digital assistants (PDAs), digital cameras, digital

11

media players, digital recorders, games, appliances, vehicles, wireless devices, mobile telephones and the like. The memory sub-system controller **115** (e.g., a controller external to the memory device **130**), may be a memory controller or other external host device.

Memory device **130** includes an array of memory cells **104** logically arranged in rows and columns. Memory cells of a logical row are typically connected to the same access line (e.g., a wordline) while memory cells of a logical column are typically selectively connected to the same data line (e.g., a bitline). A single access line may be associated with more than one logical row of memory cells and a single data line may be associated with more than one logical column. Memory cells (not shown in FIG. 1B) of at least a portion of array of memory cells **104** are capable of being programmed to one of at least two target data states.

Row decode circuitry **108** and column decode circuitry **110** are provided to decode address signals. Address signals are received and decoded to access the array of memory cells **104**. Memory device **130** also includes input/output (I/O) control circuitry **160** to manage input of commands, addresses and data to the memory device **130** as well as output of data and status information from the memory device **130**. An address register **114** is in communication with I/O control circuitry **160** and row decode circuitry **108** and column decode circuitry **110** to latch the address signals prior to decoding. A command register **124** is in communication with I/O control circuitry **160** and local media controller **135** to latch incoming commands.

A controller (e.g., the local media controller **135** internal to the memory device **130**) controls access to the array of memory cells **104** in response to the commands and generates status information for the external memory sub-system controller **115**, i.e., the local media controller **135** is configured to perform access operations (e.g., read operations, programming operations and/or erase operations) on the array of memory cells **104**. The local media controller **135** is in communication with row decode circuitry **108** and column decode circuitry **110** to control the row decode circuitry **108** and column decode circuitry **110** in response to the addresses. In one embodiment, local media controller **135** includes the PPM component **137**, which can implement the defect detection described herein during an erase operation on memory device **130**.

The local media controller **135** is also in communication with a cache register **118**. Cache register **118** latches data, either incoming or outgoing, as directed by the local media controller **135** to temporarily store data while the array of memory cells **104** is busy writing or reading, respectively, other data. During a program operation (e.g., write operation), data may be passed from the cache register **118** to the data register **170** for transfer to the array of memory cells **104**; then new data may be latched in the cache register **118** from the I/O control circuitry **160**. During a read operation, data may be passed from the cache register **118** to the I/O control circuitry **160** for output to the memory sub-system controller **115**; then new data may be passed from the data register **170** to the cache register **118**. The cache register **118** and/or the data register **170** may form (e.g., may form a portion of) a page buffer of the memory device **130**. A page buffer may further include sensing devices (not shown in FIG. 1B) to sense a data state of a memory cell of the array of memory cells **204**, e.g., by sensing a state of a data line connected to that memory cell. A status register **122** may be in communication with I/O control circuitry **160** and the local memory controller **135** to latch the status information for output to the memory sub-system controller **115**.

12

Memory device **130** receives control signals at the memory sub-system controller **115** from the local media controller **135** over a control link **132**. For example, the control signals can include a chip enable signal CE #, a command latch enable signal CLE, an address latch enable signal ALE, a write enable signal WE #, a read enable signal RE #, and a write protect signal WP #. Additional or alternative control signals (not shown) may be further received over control link **132** depending upon the nature of the memory device **130**. In one embodiment, memory device **130** receives command signals (which represent commands), address signals (which represent addresses), and data signals (which represent data) from the memory sub-system controller **115** over a multiplexed input/output (I/O) bus **136** and outputs data to the memory sub-system controller **115** over I/O bus **136**.

For example, the commands may be received over input/output (I/O) pins [7:0] of I/O bus **136** at I/O control circuitry **160** and may then be written into command register **124**. The addresses may be received over input/output (I/O) pins [7:0] of I/O bus **136** at I/O control circuitry **160** and may then be written into address register **114**. The data may be received over input/output (I/O) pins [7:0] for an 8-bit device or input/output (I/O) pins [15:0] for a 16-bit device at I/O control circuitry **160** and then may be written into cache register **118**. The data may be subsequently written into data register **170** for programming the array of memory cells **104**.

In an embodiment, cache register **118** may be omitted, and the data may be written directly into data register **170**. Data may also be output over input/output (I/O) pins [7:0] for an 8-bit device or input/output (I/O) pins [15:0] for a 16-bit device. Although reference may be made to I/O pins, they may include any conductive node providing for electrical connection to the memory device **130** by an external device (e.g., the memory sub-system controller **115**), such as conductive pads or conductive bumps as are commonly used.

It will be appreciated by those skilled in the art that additional circuitry and signals can be provided, and that the memory device **130** of FIGS. 1A-1B has been simplified. It should be recognized that the functionality of the various block components described with reference to FIGS. 1A-1B may not necessarily be segregated to distinct components or component portions of an integrated circuit device. For example, a single component or component portion of an integrated circuit device could be adapted to perform the functionality of more than one block component of FIGS. 1A-1B. Alternatively, one or more components or component portions of an integrated circuit device could be combined to perform the functionality of a single block component of FIGS. 1A-1B. Additionally, while specific I/O pins are described in accordance with popular conventions for receipt and output of the various signals, it is noted that other combinations or numbers of I/O pins (or other I/O node structures) may be used in the various embodiments.

FIGS. 2A-2C are diagrams of portions of an example array of memory cells included in a memory device, in accordance with some embodiments of the present disclosure. For example, FIG. 2A is a schematic of a portion of an array of memory cells **200A** as could be used in a memory device (e.g., as a portion of array of memory cells **104**). Memory array **200A** includes access lines, such as wordlines **202₀** to **202_N**, and a data line, such as bitline **204**. The wordlines **202** may be connected to global access lines (e.g., global wordlines), not shown in FIG. 2A, in a many-to-one relationship. For some embodiments, memory array **200A** may be formed over a semiconductor that, for example, may be conductively doped to have a conductivity type, such as

13

a p-type conductivity, e.g., to form a p-well, or an n-type conductivity, e.g., to form an n-well.

Memory array 200A can be arranged in rows each corresponding to a respective wordline 202 and columns each corresponding to a respective bitline 204. Rows of memory cells 208 can be divided into one or more groups of physical pages of memory cells 208, and physical pages of memory cells 208 can include every other memory cell 208 commonly connected to a given wordline 202. For example, memory cells 208 commonly connected to wordline 202_N and selectively connected to even bitlines 204 (e.g., bitlines 204₀, 204₂, 204₄, etc.) may be one physical page of memory cells 208 (e.g., even memory cells) while memory cells 208 commonly connected to wordline 202_N and selectively connected to odd bitlines 204 (e.g., bitlines 204₁, 204₃, 204₅, etc.) may be another physical page of memory cells 208 (e.g., odd memory cells). Although bitlines 204₃-204₅ are not explicitly depicted in FIG. 2A, it is apparent from the figure that the bitlines 204 of the array of memory cells 200A may be numbered consecutively from bitline 204₀ to bitline 204_M. Other groupings of memory cells 208 commonly connected to a given wordline 202 may also define a physical page of memory cells 208. For certain memory devices, all memory cells commonly connected to a given wordline might be deemed a physical page of memory cells. The portion of a physical page of memory cells (which, in some embodiments, could still be the entire row) that is read during a single read operation or programmed during a single programming operation (e.g., an upper or lower page of memory cells) might be deemed a logical page of memory cells. A block of memory cells may include those memory cells that are configured to be erased together, such as all memory cells connected to wordlines 202₀-202_N (e.g., all strings 206 sharing common wordlines 202). Unless expressly distinguished, a reference to a page of memory cells herein refers to the memory cells of a logical page of memory cells.

Each column can include a string of series-connected memory cells (e.g., non-volatile memory cells), such as one of strings 206₀ to 206_M. Each string 206 can be connected (e.g., selectively connected) to a source line 216 (SRC) and can include memory cells 208₀ to 208_N. The memory cells 208 of each string 206 can be connected in series between a select gate 210, such as one of the select gates 210₀ to 210_M, and a select gate 212, such as one of the select gates 212₀ to 212_M. In some embodiments, the select gates 210₀ to 210_M are source-side select gates (SGS) and the select gates 212₀ to 212_M are drain-side select gates. Select gates 210₀ to 210_M can be connected to a select line 214 (e.g., source-side select line) and select gates 212₀ to 212_M can be connected to a select line 215 (e.g., drain-side select line). The select gates 210 and 212 might represent a plurality of select gates connected in series, with each select gate in series configured to receive a same or independent control signal. A source of each select gate 210 can be connected to SRC 216, and a drain of each select gate 210 can be connected to a memory cell 208₀ of the corresponding string 206. Therefore, each select gate 210 can be configured to selectively connect a corresponding string 206 to SRC 216. A control gate of each select gate 210 can be connected to select line 214. The drain of each select gate 212 can be connected to the bitline 204 for the corresponding string 206. The source of each select gate 212 can be connected to a memory cell 208_N of the corresponding string 206. Therefore, each select gate 212 might be configured to selectively

14

connect a corresponding string 206 to the bitline 204. A control gate of each select gate 212 can be connected to select line 215.

In some embodiments, and as will be described in further detail below with reference to FIG. 2B, the memory array in FIG. 2A is a three-dimensional memory array, in which the strings 206 extend substantially perpendicular to a plane containing SRC 216 and to a plane containing a plurality of bitlines 204 that can be substantially parallel to the plane containing SRC 216.

FIG. 2B is another schematic of a portion of an array of memory cells 200B (e.g., a portion of the array of memory cells 104) arranged in a three-dimensional memory array structure. The three-dimensional memory array 200B may incorporate vertical structures which may include semiconductor pillars where a portion of a pillar may act as a channel region of the memory cells of strings 206. The strings 206 may be each selectively connected to a bit line 204₀-204_M by a select gate 212 and to the SRC 216 by a select gate 210. Multiple strings 206 can be selectively connected to the same bitline 204. Subsets of strings 206 can be connected to their respective bitlines 204 by biasing the select lines 215₀-215_L to selectively activate particular select gates 212 each between a string 206 and a bitline 204. The select gates 210 can be activated by biasing the select line 214. Each wordline 202 may be connected to multiple rows of memory cells of the memory array 200B. Rows of memory cells that are commonly connected to each other by a particular wordline 202 may collectively be referred to as tiers.

FIG. 2C is a diagram of a portion of an array of memory cells 200C (e.g., a portion of the array of memory cells 104). Channel regions (e.g., semiconductor pillars) 238₀₀ and 238₀₁ represent the channel regions of different strings of series-connected memory cells (e.g., strings 206 of FIGS. 2A-2B) selectively connected to the bitline 204₀. Similarly, channel regions 238₁₀ and 238₁₁ represent the channel regions of different strings of series-connected memory cells (e.g., NAND strings 206 of FIGS. 2A-2B) selectively connected to the bitline 204₁. A memory cell (not depicted in FIG. 2C) may be formed at each intersection of a wordline 202 and a channel region 238, and the memory cells corresponding to a single channel region 238 may collectively form a string of series-connected memory cells (e.g., a string 206 of FIGS. 2A-2B). Additional features might be common in such structures, such as dummy wordlines, segmented channel regions with interposed conductive regions, etc.

FIG. 3 is a block diagram illustrating a multi-die package 300 with multiple memory dies in a memory sub-system, in accordance with some embodiments of the present disclosure. As illustrated, multi-die package 300 includes memory dies 330(0)-330(7). In other embodiments, however, multi-die package 300 can include some other number of memory dies, such as additional or fewer memory dies. In one embodiment, memory dies 330(0)-330(7) share a clock signal ICLK which is received via a clock signal line. Memory dies 330(0)-330(7) can be selectively enabled in response to a chip enable signal (e.g. via a control link), and can communicate over a separate I/O bus. In addition, a peak current magnitude indicator signal HC # is commonly shared between the memory dies 330(0)-330(7). The peak current magnitude indicator signal HC # can be normally pulled to a particular state (e.g., pulled high). In one embodiment, each of memory dies 330(0)-330(7) includes an instance of PPM component 137, which receives both the clock signal ICLK and the peak current magnitude indicator signal HC #.

In one embodiment, a token-based protocol is used where a token cycles through each of the memory dies 330(0)-330(7) for determining and broadcasting expected peak current magnitude, even though some of the memory dies 330(0)-330(7) might be disabled in response to their respective chip enable signal. The period of time during which a given PPM component 137 holds this token (e.g. a certain number of cycles of clock signal ICLK) can be referred to herein as a power management cycle of the associated memory die. At the end of the power management cycle, the token is passed to another memory die. Eventually the token is received again by the same PPM component 137, which signals the beginning of the power management cycle for the associated memory die. In one embodiment, the encoded value for the lowest expected peak current magnitude is configured such that each of its digits correspond to the normal logic level of the peak current magnitude indicator signal HC # where the disabled dies do not transition the peak current magnitude indicator signal HC #. In other embodiments, however, the memory dies can be configured, when otherwise disabled in response to their respective chip enable signal, to drive transitions of the peak current magnitude indicator signal HC # to indicate the encoded value for the lowest expected peak current magnitude upon being designated. When a given PPM component 137 holds the token, it can determine the peak current magnitude for the respective one of memory die 330(0)-330(7), which can be attributable to one or more processing threads on that memory die, and broadcast an indication of the same via the peak current magnitude indicator signal HC #. During a given power management cycle, the PPM component 137 can arbitrate among the multiple processing threads on the respective memory die using one of a number of different arbitration schemes in order to allocate that peak current to enable concurrent memory access operations.

FIG. 4 is a block diagram illustrating a multi-plane memory device 130 configured for independent parallel plane access, in accordance with some embodiments of the present disclosure. The memory planes 472(0)-472(3) can each be divided into blocks of data, with a different relative block of data from two or more of the memory planes 472(0)-472(3) concurrently accessible during memory access operations. For example, during memory access operations, two or more of data block 482 of the memory plane 472(0), data block 483 of the memory plane 472(1), data block 484 of the memory plane 372(2), and data block 485 of the memory plane 4372(3) can each be accessed concurrently. For example, memory planes 472(0)-472(3) can correspond to a portion of memory array 104 of FIG. 1B.

The memory device 130 includes a memory array 470 divided into memory planes 472(0)-472(3) that each includes a respective number of memory cells. The multi-plane memory device 130 can further include local media controller 135, including a power control circuit and access control circuit for concurrently performing memory access operations for different memory planes 472(0)-472(3). The memory cells can be non-volatile memory cells, such as NAND flash cells, or can generally be any type of memory cells.

The memory planes 472(0)-472(3) can each be divided into blocks of data, with a different relative block of data from each of the memory planes 472(0)-472(3) concurrently accessible during memory access operations. For example, during memory access operations, data block 482 of the memory plane 472(0), data block 383 of the memory plane

472(1), data block 484 of the memory plane 472(2), and data block 385 of the memory plane 472(3) can each be accessed concurrently.

Each of the memory planes 472(0)-472(3) can be coupled to a respective page buffer 476(0)-476(3). Each page buffer 476(0)-476(3) can be configured to provide data to or receive data from the respective memory plane 472(0)-472(3). The page buffers 476(0)-476(3) can be controlled by local media controller 135. Data received from the respective memory plane 472(0)-472(3) can be latched at the page buffers 476(0)-476(3), respectively, and retrieved by local media controller 135, and provided to the memory sub-system controller 115 via the interface.

Each of the memory planes 472(0)-472(3) can be further coupled to a respective access driver circuit 474(0)-474(3), such as an access line driver circuit. The driver circuits 474(0)-474(3) can be configured to condition a page of a respective block of an associated memory plane 472(0)-472(3) for a memory access operation, such as programming data (i.e., writing data), reading data, or erasing data. Each of the driver circuits 474(0)-474(3) can be coupled to a respective global access lines associated with a respective memory plane 472(0)-472(3). Each of the global access lines can be selectively coupled to respective local access lines within a block of a plane during a memory access operation associated with a page within the block. The driver circuits 474(0)-474(3) can be controlled based on signals from local media controller 135. Each of the driver circuits 474(0)-474(3) can include or be coupled to a respective power circuit, and can provide voltages to respective access lines based on voltages provided by the respective power circuit. The voltages provided by the power circuits can be based on signals received from local media controller 135.

The local media controller 135 can control the driver circuits 474(0)-474(3) and page buffers 476(0)-476(3) to concurrently perform memory access operations associated with each of a group of memory command and address pairs (e.g., received from memory sub-system controller 115). For example, local media controller 135 can control the driver circuits 474(0)-474(3) and page buffer 476(0)-476(3) to perform the concurrent memory access operations. Local media controller 135 can include a power control circuit that serially configures two or more of the driver circuits 474(0)-474(3) for the concurrent memory access operations, and an access control circuit configured to control two or more of the page buffers 476(0)-476(3) to sense and latch data from the respective memory planes 472(0)-472(3), or program data to the respective memory planes 472(0)-472(3) to perform the concurrent memory access operations.

In operation, local media controller 135 can receive a group of memory command and address pairs via the bus, with each pair arriving in parallel or serially. In some examples, the group of memory command and address pairs can each be associated with different respective memory planes 472(0)-472(3) of the memory array 470. The local media controller 135 can be configured to perform concurrent memory access operations (e.g., read operations or program operations) for the different memory planes 472(0)-472(3) of the memory array 470 responsive to the group of memory command and address pairs. For example, the power control circuit of local media controller 135 can serially configure, for the concurrent memory access operations based on respective page type (e.g., UP, TP, LP, XP, SLC/MLC/TLC/QLC page), the driver circuits 474(0)-474(3) for two or more memory planes 472(0)-472(3) associated with the group of memory command and address pairs. After the access line driver circuits 474(0)-474(3) have been

configured, the access control circuit of local media controller 135 can concurrently control the page buffers 476(0)-476(3) to access the respective pages of each of the two or more memory planes 472(0)-472(3) associated with the group of memory command and address pairs, such as retrieving data or writing data, during the concurrent memory access operations. For example, the access control circuit can concurrently (e.g., in parallel and/or contemporaneously) control the page buffers 476(0)-476(3) to charge/discharge bitlines, sense data from the two or more memory planes 472(0)-472(3), and/or latch the data.

Based on the signals received from local media controller 135, the driver circuits 474(0)-474(3) that are coupled to the memory planes 472(0)-472(3) associated with the group of memory command and address command pairs can select blocks of memory or memory cells from the associated memory plane 472(0)-472(3), for memory operations, such as read, program, and/or erase operations. The driver circuits 474(0)-474(3) can drive different respective global access lines associated with a respective memory plane 472(0)-472(3). As an example, the driver circuit 474(0) can drive a first voltage on a first global access line associated with the memory plane 472(0), the driver circuit 474(1) can drive a second voltage on a third global access line associated with the memory plane 472(1), the driver circuit 474(2) can drive a third voltage on a seventh global access line associated with the memory plane 472(2), etc., and other voltages can be driven on each of the remaining global access lines. In some examples, pass voltages can be provided on all access lines except an access line associated with a page of a memory plane 472(0)-472(3) to be accessed. The local media controller 135, the driver circuits 474(0)-474(3) can allow different respective pages, and the page buffers 476(0)-476(3) within different respective blocks of memory cells, to be accessed concurrently. For example, a first page of a first block of a first memory plane can be accessed concurrently with a second page of a second block of a second memory plane, regardless of page type.

The page buffers 476(0)-476(3) can provide data to or receive data from the local media controller 135 during the memory access operations responsive to signals from the local media controller 135 and the respective memory planes 472(0)-472(3). The local media controller 135 can provide the received data to memory sub-system controller 115.

It will be appreciated that the memory device 130 can include more or less than four memory planes, driver circuits, and page buffers. It will also be appreciated that the respective global access lines can include 8, 16, 32, 64, 128, etc., global access lines. The local media controller 135 and the driver circuits 474(0)-474(3) can concurrently access different respective pages within different respective blocks of different memory planes when the different respective pages are of a different page type. For example, local media controller 135 can include a number of different processing threads, such as processing threads 434(0)-434(3). Each of processing threads 434(0)-434(3) can be associated with a respective one of memory planes 472(0)-472(3), or a respective group of memory planes, and can manage operations performed on the respective plane or group of planes. For example, each of processing threads 434(0)-434(3) can provide control signals to the respective one of driver circuits 474(0)-474(3) and page buffers 476(0)-476(3) to perform those memory access operations concurrently (e.g., at least partially overlapping in time). Since the processing threads 434(0)-434(3) can perform the memory access operations, each of processing threads 434(0)-434(3) can have different current requirements at different points in

time. PPM component 137 can determine the power budget needs of processing threads 434(0)-434(3) in a given power management cycle and identify one or more of processing threads 434(0)-434(3) using one of a number of power budget arbitration schemes described herein. The one or more processing threads 434(0)-434(3) can be determined based on an available power budget in the memory sub-system 110 during the power management cycles. For example, PPM component 137 can determine respective priorities of processing threads 434(0)-434(3), and allocate current to processing threads 434(0)-434(3) based on the respective priorities.

FIG. 5 is a diagram 500 of an example request to perform an operation, in accordance with some embodiments of the present disclosure. In this illustrative embodiment, the request includes a prefix command. However, such an example should not be considered limiting.

As shown, the diagram 500 depicts a cycle type 510 for a corresponding request 520. For example, the request 520 can be received, from a memory sub-system controller, at I/O control circuitry over I/O pins of an I/O bus (e.g., received, from memory sub-system controller 115, at I/O control circuitry 160 over I/O pins of the I/O bus 136 of FIG. 1B). In some embodiments, the request 520 can be received over I/O pins [7:0] for an 8-bit device. In some embodiments, the request 520 can be received over I/O pins [15:0] for a 16-bit device. The diagram 500 shows that the request 520 includes a prefix 512 having a value 522, a command 514-1 having a value 524-1, an address section 516 having respective values 526 including C1, C2, R1, R2, R3, R4, a command 514-2 having a value 524-2, and a data output section 518 having respective values 528 including D_n , D_{n+1} and D_{n+2} . For example, the operation can be a read page operation having value 524-1 set as 00h and value 524-2 set as 30h.

The value 514 can indicate whether the request 520 is a priority command to execute a priority operation with PPM priority override. That is, the prefix 512 can indicate whether PPM priority should be enabled for the command 514. The prefix can specify a particular page. A first set of prefixes can include a number of prefixes that each indicate that the operation is a non-priority operation addressing a respective location. A second set of prefixes can include a number of prefixes that each indicate that the operation is a priority operation addressing respective page.

For example, for a triple level cell (TLC) implementation in which there are three pages (e.g., lower page (LP), upper page (UP) and extra page (XP)), the first set of prefixes and the second set of prefixes can each include three prefixes. The first set of prefixes can include a 01h prefix for the LP address, a 02h prefix for the UP address, and a 03h prefix for the XP address (e.g., the first bit indicates that the operation is a non-priority operation, and the second bit indicates the page). The second set of prefixes can include a 41h prefix for the LP address, a 42h prefix for the UP address, and a 43h prefix for the XP address (e.g., the first bit indicates that the operation is a priority operation to be executed with PPM priority override, and the second bit indicates the page). It is to be understood and appreciated that the prefix values described herein are purely exemplary.

FIG. 6 is a timing diagram ("diagram") 600 of an example execution of a priority operation, in accordance with some embodiments of the present disclosure. In this illustrative example, the priority operation is a read operation. However, such an example should not be considered limiting. For example, the priority operation can be a program operation or an erase operation.

The priority operation can include a number of sub-operations, including a prologue sub-operation, a read initialization sub-operation, a sensing sub-operation and a read recovery sub-operation. The diagram 600 shows current consumption over time during the priority operation. The priority operation can be broken up into a number of time periods each corresponding to a respective one of the sub-operations. For example, the time periods can include a prologue time period ($t_{prologue}$) 610, a read initialization time period (t_{read_init}) 620, a sensing time period (t_{sense}) 630, and a read recovery time period (t_{rcv}) 640.

For example, during the prologue time period 610, the local media controller (e.g., local media controller 135 of FIGS. 1A-1B) may activate voltage pumps (e.g., cause voltage pumps to be turned on) and load information for the read operation. During the read initialization time period 620, the local media controller may cause voltages to be set (e.g., wordline voltage ramping) and may cause bitline precharge. During the sensing time period 630, the local media controller may cause a target wordline to be set to a target read voltage, and may cause the cells connected to the target wordline to be sensed to determine if the threshold voltage is above or below the target read voltage. For a memory device including cells having multiple bits per cell (e.g., MLC, TLC, QLC, PLC), multiple read levels may be applied to the target wordline. During the read recovery time period 640, the local media controller may cause the cells to discharge, may deactivate the voltage pumps (e.g., cause the voltage pumps to be turned off) and may cause the memory device to return to an idle or standby state (e.g., depending on the state of the CE # signal).

The boundaries of each time period and/or sub-period are defined by respective pairs of breakpoints. For example, as shown, the prologue time period 610 is defined between breakpoints 650-1 and 650-2, the read initialization time period 620 is defined between breakpoints 650-2 and 650-3, the sensing time period 630 is defined between breakpoints 650-3 and 650-4, and the read recovery time period 640 is defined between breakpoints 650-4 and 650-5.

In the case that PPM priority override is not enabled, the breakpoints 650-1 and 650-2 are treated as HC breakpoints, while the other breakpoints 650-3 through 650-5 are treated as LC breakpoints. The HC breakpoints can force the memory device (e.g., die) to wait for sufficient current budget to be made available to overcome the HC breakpoints for executing respective sub-operations (e.g., initiate the prologue sub-operation and initiate the read initialization sub-operation after the prologue sub-operation). For example, the local media controller may first request confirmation from the PPM component that there is sufficient current budget to initiate execution of a sub-operation gated by a HC breakpoint. If not, the local media controller may have to wait for additional current budget to be made available, or other dies may have execution of their operations suspended and/or paused to release current budget for the memory device to execute the priority operation.

In the case of PPM priority override being enabled (e.g., the request indicates that PPM priority override is to be enabled (e.g., prefix command) or PPM priority override is automatically enabled), the breakpoints 650-1 and 650-2 can be reconfigured as LC breakpoints. The LC breakpoints enable the memory device to initiate the prologue and read initialization sub-operations regardless of the amount of available current budget. Thus, enabling PPM priority override can enable the memory device to execute the priority operation without having to wait for additional current

budget, or having to suspend and/or pause the execution of other operations performed by other memory devices.

FIG. 7 is a graph 700 illustrating current budget over time, in accordance with some embodiments of the present disclosure. The graph 700 includes an x-axis indicating time (t) 710 and a y-axis indicating quantized current (i) 720. The graph 700 shows the total system quantized current being consumed ("system current") 730 and the PPM quantized current budget ("PPM budget") 740 over time. As shown, the PPM budget 740 can be held constant. As mentioned above, priority operations can cause the system current 730 to exceed the PPM budget 740. For example, the graph 700 further shows a box 750, which indicates a period of time in which the system current 730 exceeds the PPM budget 740. This condition can remain until the system can reduce the system current 730 to be less than or equal to the PPM budget 740 (e.g., the system can pause enough HC states to sufficiently reduce the system current 730).

FIG. 8A is a flow diagram of a method 800 to implement peak power management (PPM) in a memory device, in accordance with some embodiments of the present disclosure. The method 800 can be performed by processing logic that can include hardware (e.g., processing device, circuitry, dedicated logic, programmable logic, microcode, hardware of a device, integrated circuit, etc.), software (e.g., instructions run or executed on a processing device), or a combination thereof. In some embodiments, the method 800 is performed by the local media controller 135 and/or the PPM component 137 of FIGS. 1A-1B. Although shown in a particular sequence or order, unless otherwise specified, the order of the processes can be modified. Thus, the illustrated embodiments should be understood only as examples, and the illustrated processes can be performed in a different order, and some processes can be performed in parallel. Additionally, one or more processes can be omitted in various embodiments. Thus, not all processes are required in every embodiment. Other process flows are possible.

At block 810, a request to perform an operation is received. For example, processing logic can receive the request to perform the operation with respect to a memory device, such as memory device 130. In some embodiments, the memory device 130 is a die of a multi-die memory sub-system, such as memory sub-system 110. The request can be received from a memory sub-system controller, such as memory sub-system controller 115. The processing logic can communicate with PPM components of other dies of the memory device to implement PPM via a token communication bus. In some embodiments, the operation is a read operation (e.g., snap read or full page read). For example, the request can be similar to the request of diagram 500 described above with reference to FIG. 5.

At block 820, it is determined whether to initiate a PPM priority override procedure. For example, the processing logic can determine whether to enable the PPM priority override procedure. In some embodiments, PPM priority override is automatically enabled when PPM is active. In some embodiments, determining whether to enable the PPM priority override procedure includes determining whether the operation is a priority operation to be executed with PPM priority override. For example, determining whether the operation is a priority operation can include determining whether the request is a priority command.

In some embodiments, a priority command includes a prefix indicating that the operation is a priority operation to be executed with PPM priority override. The prefix can include a bit specifying whether the operation is a priority operation to be executed with PPM priority override, and a

bit specifying a particular location within the memory device (e.g., page). For example, for a triple level cell (TLC) implementation in which there are three pages (e.g., lower page (LP), upper page (UP) and extra page (XP)), three prefixes can be defined that each indicate that the operation is a priority operation with respect to a respective page.

If it is determined not to initiate a PPM priority override procedure, this means that the operation is treated as a non-priority operation. At block **830**, the operation is handled as a non-priority operation. For example, the processing logic can execute the operation without reconfiguring any HC breakpoints. For each HC breakpoint, the processing logic can determine, from the PPM component, whether there is sufficient available current within the system to proceed with the operation. If there is sufficient available current, then the PPM component will wait until it is in possession of the token, communicate an updated (e.g., higher) current consumption value with the other memory devices (e.g., via their respective PPM components), and notify the processing logic that it can proceed with executing the operation.

In the event that there is insufficient available current to proceed, the processing logic can receive a notification from the PPM component that there is insufficient current budget. In response, the processing logic can pause or suspend the operation. The PPM component can report a new idle current consumption value to the other memory devices (e.g., via their respective PPM components) and wait until there is sufficient available current to proceed. Thus, if the operation is being handled as a non-priority operation, then the processing logic can, at each HC breakpoint, wait until an amount of available current budget is sufficient to proceed during execution of the operation. At each LC breakpoint, the processing logic can proceed with execution of the operation. For example, the PPM component will wait until it is in possession of the token and will report the updated (lower) current consumption value at each LC breakpoint to the other memory devices (e.g., via their respective PPM component).

However, if it is determined to initiate a PPM priority override procedure, this means that the operation is treated as a priority operation and that the processing logic need not wait for current budget to be made available to execute the operation. At block **840**, the PPM override procedure is performed. For example, the processing logic can cause the PPM override procedure to be performed. For example, the PPM override procedure can be similar to that shown in diagram **600** described above with reference to FIG. **6**. Further details regarding performing the PPM override procedure will now be described below with reference to FIG. **8B**.

FIG. **8B** is a flow diagram of a method **840** to perform the PPM override procedure of FIG. **8A**, in accordance with some embodiments of the present disclosure. The method **840** can be performed by processing logic that can include hardware (e.g., processing device, circuitry, dedicated logic, programmable logic, microcode, hardware of a device, integrated circuit, etc.), software (e.g., instructions run or executed on a processing device), or a combination thereof. In some embodiments, the method **840** is performed by the local media controller **135** and/or the PPM component **137** of FIGS. **1A-1B**. Although shown in a particular sequence or order, unless otherwise specified, the order of the processes can be modified. Thus, the illustrated embodiments should be understood only as examples, and the illustrated processes can be performed in a different order, and some processes can be performed in parallel. Additionally, one or

more processes can be omitted in various embodiments. Thus, not all processes are required in every embodiment. Other process flows are possible.

To perform the PPM override procedure in order to execute the operation as a priority operation, at block **842**, breakpoint reconfiguration is performed. For example, the processing logic can cause each HC breakpoint defined for the operation to be reconfigured as a respective LC breakpoints. Reconfiguring each HC breakpoint as a respective LC breakpoint serves to remove the gating mechanism that had previously prevented operations from being executed without receiving confirmation that there is sufficient available current budget. In some embodiments, the operation defines a pair of HC breakpoints, where one HC breakpoint of the pair of HC breakpoints serves as a gate to initiate the performance of an initial sub-operation of the operation (e.g., before executing a prologue sub-operation of a read operation), and another HC breakpoint of the pair of HC breakpoints can serve as a gate to initiate the performance of a subsequent sub-operation of the operation (e.g., before executing a read initialization sub-operation after completion of the prologue sub-operation).

At operation **844**, updated current consumption is reported. For example, during execution of the operation, the processing logic can communicate, at the breakpoint, a respective updated current consumption value for executing the sub-operation to the PPM component of the memory device. The PPM component can report the respective updated current consumption value to other memory devices of the system (e.g., via their respective PPM components).

At operation **846**, execution of the operation is continued. For example, the processing logic can cause execution of the sub-operation. Since the HC breakpoints have been reconfigured as LC breakpoints, execution of the sub-operation can proceed without interruption. For example, execution of the operation can proceed without pausing or suspending execution of the operation while waiting for current budget to be made available.

At operation **848**, it is determined whether the breakpoint is the final breakpoint. For example, the processing logic can determine whether another breakpoint defining another sub-operation is to be executed. If the breakpoint is not the final breakpoint, this means that execution of the operation is not complete and the process can revert back to operation **844** to report the updated current consumption with respect to the next breakpoint. If the breakpoint is the final breakpoint (e.g., the breakpoint before the read recovery sub-operation, this means that execution of the operation is complete, and the procedure terminates. Further details regarding operations **810-870** are described above with reference to FIGS. **1** and **5-8**.

FIG. **9** illustrates an example machine of a computer system **900** within which a set of instructions, for causing the machine to perform any one or more of the methodologies discussed herein, can be executed. In some embodiments, the computer system **900** can correspond to a host system (e.g., the host system **120** of FIG. **1A**) that includes, is coupled to, or utilizes a memory sub-system (e.g., the memory sub-system **110** of FIG. **1A**) or can be used to perform the operations of a controller (e.g., to execute an operating system to perform operations corresponding to the local media controller **135** and/or the PPM component **137** of FIG. **1A**). In alternative embodiments, the machine can be connected (e.g., networked) to other machines in a LAN, an intranet, an extranet, and/or the Internet. The machine can operate in the capacity of a server or a client machine in client-server network environment, as a peer machine in a

peer-to-peer (or distributed) network environment, or as a server or a client machine in a cloud computing infrastructure or environment.

The machine can be a personal computer (PC), a tablet PC, a set-top box (STB), a Personal Digital Assistant (PDA), a memory cellular telephone, a web appliance, a server, a network router, a switch or bridge, or any machine capable of executing a set of instructions (sequential or otherwise) that specify actions to be taken by that machine. Further, while a single machine is illustrated, the term “machine” shall also be taken to include any collection of machines that individually or jointly execute a set (or multiple sets) of instructions to perform any one or more of the methodologies discussed herein.

The example computer system 900 includes a processing device 902, a main memory 1004 (e.g., read-only memory (ROM), flash memory, dynamic random access memory (DRAM) such as synchronous DRAM (SDRAM) or RDRAM, etc.), a static memory 906 (e.g., flash memory, static random access memory (SRAM), etc.), and a data storage system 918, which communicate with each other via a bus 930.

Processing device 902 represents one or more general-purpose processing devices such as a microprocessor, a central processing unit, or the like. More particularly, the processing device can be a complex instruction set computing (CISC) microprocessor, reduced instruction set computing (RISC) microprocessor, very long instruction word (VLIW) microprocessor, or a processor implementing other instruction sets, or processors implementing a combination of instruction sets. Processing device 902 can also be one or more special-purpose processing devices such as an application specific integrated circuit (ASIC), a field programmable gate array (FPGA), a digital signal processor (DSP), network processor, or the like. The processing device 902 is configured to execute instructions 926 for performing the operations and steps discussed herein. The computer system 900 can further include a network interface device 908 to communicate over the network 920.

The data storage system 918 can include a machine-readable storage medium 924 (also known as a computer-readable medium) on which is stored one or more sets of instructions 926 or software embodying any one or more of the methodologies or functions described herein. The instructions 926 can also reside, completely or at least partially, within the main memory 904 and/or within the processing device 902 during execution thereof by the computer system 900, the main memory 904 and the processing device 902 also constituting machine-readable storage media. The machine-readable storage medium 924, data storage system 918, and/or main memory 904 can correspond to the memory sub-system 110 of FIG. 1A.

In one embodiment, the instructions 926 include instructions to implement functionality corresponding to a local media controller and/or PPM component (e.g., the local media controller 135 and/or the PPM component 137 of FIG. 1A). While the machine-readable storage medium 924 is shown in an example embodiment to be a single medium, the term “machine-readable storage medium” should be taken to include a single medium or multiple media that store the one or more sets of instructions. The term “machine-readable storage medium” shall also be taken to include any medium that is capable of storing or encoding a set of instructions for execution by the machine and that cause the machine to perform any one or more of the methodologies of the present disclosure. The term “machine-readable storage medium” shall accordingly be

taken to include, but not be limited to, solid-state memories, optical media, and magnetic media.

Some portions of the preceding detailed descriptions have been presented in terms of algorithms and symbolic representations of operations on data bits within a computer memory. These algorithmic descriptions and representations are the ways used by those skilled in the data processing arts to most effectively convey the substance of their work to others skilled in the art. An algorithm is here, and generally, conceived to be a self-consistent sequence of operations leading to a desired result. The operations are those requiring physical manipulations of physical quantities. Usually, though not necessarily, these quantities take the form of electrical or magnetic signals capable of being stored, combined, compared, and otherwise manipulated. It has proven convenient at times, principally for reasons of common usage, to refer to these signals as bits, values, elements, symbols, characters, terms, numbers, or the like.

It should be borne in mind, however, that all of these and similar terms are to be associated with the appropriate physical quantities and are merely convenient labels applied to these quantities. The present disclosure can refer to the action and processes of a computer system, or similar electronic computing device, that manipulates and transforms data represented as physical (electronic) quantities within the computer system’s registers and memories into other data similarly represented as physical quantities within the computer system memories or registers or other such information storage systems.

The present disclosure also relates to an apparatus for performing the operations herein. This apparatus can be specially constructed for the intended purposes, or it can include a general purpose computer selectively activated or reconfigured by a computer program stored in the computer. Such a computer program can be stored in a computer readable storage medium, such as any type of disk including floppy disks, optical disks, CD-ROMs, and magnetic-optical disks, read-only memories (ROMs), random access memories (RAMs), EPROMs, EEPROMs, magnetic or optical cards, or any type of media suitable for storing electronic instructions, each coupled to a computer system bus.

The algorithms and displays presented herein are not inherently related to any particular computer or other apparatus. Various general purpose systems can be used with programs in accordance with the teachings herein, or it can prove convenient to construct a more specialized apparatus to perform the method. The structure for a variety of these systems will appear as set forth in the description below. In addition, the present disclosure is not described with reference to any particular programming language. It will be appreciated that a variety of programming languages can be used to implement the teachings of the disclosure as described herein.

The present disclosure can be provided as a computer program product, or software, that can include a machine-readable medium having stored thereon instructions, which can be used to program a computer system (or other electronic devices) to perform a process according to the present disclosure. A machine-readable medium includes any mechanism for storing information in a form readable by a machine (e.g., a computer). In some embodiments, a machine-readable (e.g., computer-readable) medium includes a machine (e.g., a computer) readable storage medium such as a read only memory (“ROM”), random access memory (“RAM”), magnetic disk storage media, optical storage media, flash memory components, etc.

25

In the foregoing specification, embodiments of the disclosure have been described with reference to specific example embodiments thereof. It will be evident that various modifications can be made thereto without departing from the broader spirit and scope of embodiments of the disclosure as set forth in the following claims. The specification and drawings are, accordingly, to be regarded in an illustrative sense rather than a restrictive sense.

What is claimed is:

1. A memory device comprising:

a plurality of memory dies comprising a memory die, the memory die comprising:

a memory array; and

control logic, operatively coupled with the memory array, to perform peak power management (PPM) operations comprising:

identifying an operation to be executed with PPM priority override, wherein the operation is associated with a plurality of high current breakpoints, and wherein each high current breakpoint of the plurality of high current breakpoints corresponds to a point in time during execution of the operation at which more current is to be reserved to continue execution of the operation; and

performing a PPM priority override procedure to execute the operation with PPM priority override, wherein performing the PPM priority override procedure comprises reconfiguring each high current breakpoint of the plurality of high current breakpoints as a respective low current breakpoint of a plurality of low current breakpoints to execute the operation with PPM priority override, and wherein each low current breakpoint of the plurality of low current breakpoints corresponds to a point in time during execution of the operation at which no more current is to be reserved to continue execution of the operation.

2. The memory device of claim 1, wherein identifying the operation to be executed with PPM priority override comprises receiving a request to perform the operation to be executed with PPM priority override.

3. The memory device of claim 2, wherein identifying the operation to be executed with PPM priority override comprises determining whether the request comprises a prefix command to perform the operation to be executed with PPM priority override, and wherein the prefix command comprises a prefix indicating whether the operation is to be executed with PPM priority override.

4. The memory device of claim 3, wherein the prefix comprises a first bit specifying whether the operation is to be executed with PPM priority override, and a second bit specifying a location within the memory device.

5. The memory device of claim 1, wherein performing PPM override to execute the operation with PPM priority override further comprises causing an updated current consumption to be reported to other memory dies of the plurality of memory dies.

6. The memory device of claim 1, wherein the plurality of high current breakpoints comprises a first high current breakpoint that serves as a gate to initiate performance of an initial sub-operation of the operation, and a second high current breakpoint that serves as a gate to initiate performance of a subsequent sub-operation of the operation.

7. The memory device of claim 6, wherein the operation comprises a read operation, wherein the initial sub-operation

26

comprises a prologue sub-operation, and wherein the subsequent sub-operation comprises a read initialization sub-operation.

8. A method for performing peak power management (PPM) operations, the method comprising:

identifying, by a processing device operatively coupled to a memory die of a plurality of memory dies of a memory device, an operation to be executed with PPM priority override, wherein the operation is associated with a plurality of high current breakpoints, and wherein each high current breakpoint of the plurality of high current breakpoints corresponds to a point in time during execution of the operation at which more current is to be reserved to continue execution of the operation; and

performing, by the processing device, a PPM priority override procedure to execute the operation with PPM priority override, wherein performing the PPM priority override procedure comprises reconfiguring each high current breakpoint of the plurality of high current breakpoints as a respective low current breakpoint of a plurality of low current breakpoints to execute the operation with PPM priority override, and wherein each low current breakpoint of the plurality of low current breakpoints corresponds to a point in time during execution of the operation at which no more current is to be reserved to continue execution of the operation.

9. The method of claim 8, wherein identifying the operation to be executed with PPM priority override comprises receiving a request to perform the operation to be executed with PPM priority override.

10. The method of claim 9, wherein identifying the operation to be executed with PPM priority override comprises determining whether the request comprises a prefix command to perform the operation to be executed with PPM priority override, and wherein the prefix command comprises a prefix indicating whether the operation is to be executed with PPM priority override.

11. The method of claim 10, wherein the prefix comprises a first bit specifying whether the operation is to be executed with PPM priority override, and a second bit specifying a location within the memory device.

12. The method of claim 8, wherein performing PPM override to execute the operation with PPM priority override further comprises causing an updated current consumption to be reported to other memory dies of the plurality of memory dies.

13. The method of claim 8, wherein the plurality of high current breakpoints comprises a first high current breakpoint that serves as a gate to initiate performance of an initial sub-operation of the operation to be executed with PPM priority override, and a second high current breakpoint that serves as a gate to initiate performance of a subsequent sub-operation of the operation to be executed with PPM priority override.

14. The method of claim 13, wherein the operation comprises a read operation, wherein the initial sub-operation comprises a prologue sub-operation, and wherein the subsequent sub-operation comprises a read initialization sub-operation.

15. A non-transitory computer-readable storage medium comprising instructions that, when executed by a processing device operatively coupled to a memory die of a plurality of memory dies of a memory device, cause the processing device to perform peak power management (PPM) operations comprising:

27

receiving a request;

determining that the request comprises an operation to be executed with PPM priority override, wherein the priority operation is associated with a plurality of high current breakpoints, and wherein each high current breakpoint of the plurality of high current breakpoints corresponds to a point in time during execution of the operation at which more current is to be reserved to continue execution of the operation; and

performing the PPM priority override procedure to execute the operation with PPM priority override, wherein performing the PPM priority override comprises reconfiguring each high current breakpoint of the plurality of high current breakpoints as a respective low current breakpoint of a plurality of low current breakpoints to execute the operation with PPM priority override, and wherein each low current breakpoint of the plurality of low current breakpoints corresponds to a point in time during execution of the operation at which no more current is to be reserved to continue execution of the operation.

16. The non-transitory computer-readable storage medium of claim 15, wherein determining that the request comprises an operation to be executed with PPM priority override comprises determining whether that the request comprises a prefix command, and wherein the prefix com-

28

mand comprises a prefix indicating that the request comprises the operation to be executed with PPM priority override.

17. The non-transitory computer-readable storage medium of claim 16, wherein the prefix comprises a first bit specifying whether the request comprises the operation to be executed with PPM priority override, and a second bit specifying a location within the memory device.

18. The non-transitory computer-readable storage medium of claim 15, wherein performing PPM override to execute the operation with PPM priority override further comprises causing an updated current consumption to be reported to other memory dies of the plurality of memory dies.

19. The non-transitory computer-readable storage medium of claim 15, wherein the plurality of high current breakpoints comprises a first high current breakpoint that serves as a gate to initiate performance of an initial sub-operation of the operation to be executed with PPM priority override, and a second high current breakpoint that serves as a gate to initiate performance of a subsequent sub-operation of the operation to be executed with PPM priority override.

20. The non-transitory computer-readable storage medium of claim 19, wherein the operation comprises a read operation, wherein the initial sub-operation comprises a prologue sub-operation, and wherein the subsequent sub-operation comprises a read initialization sub-operation.

* * * * *